Tema 2

Bases de datos relacionales

Base de Datos

Tech Hub <FP>

euroformac

ÍNDICE

Objetivos de la unidad 4
1. Introducción 5
2. Modelo de datos 6
2.1 Principales características de los modelos de datos 7
2.2 Diseño de un Modelo de Datos 8
3. Terminología del Modelo Relacional 12
3.1 Relaciones, atributos, tuplas 12
3.2 Principales características de las relaciones 14
3.3 Principales características de los Atributos 15
3.4 Principales características de las tuplas 16
3.5 Principales características de las entidades o tablas 16
4. Tipos de datos 17
4.1 Introducción a los tipos de datos 17
4.2 Principales tipos de datos en MySQL 18
4.3 Principales tipos de datos en Oracle 21
4.4 Principales tipos de datos en SQL Server: 24
4.5 Principales tipos de datos en DB2 27
5. Claves primarias 30
5.1 Concepto de clave primaria 30
5.2 Claves primarias simples vs claves primarias compuestas 31
5.3 Ejemplos de claves primarias 32
6. Índices 35
6.1 Concepto de índice 35
6.2 Tipos de índices 35
6.3 Características de los índices 37
7. El valor NULL 40
7.1 Concepto de valor NULL 40
7.2 Ejemplos de valores NULL 40
8. Claves foráneas 42

8.1 Concepto de calve foránea 42
8.2 Características de las claves foráneas 43
8.3 Ejemplos de claves foráneas 43
9. Vistas 46
9.1 Concepto de vista 46
9.2 Características de las vistas 46
9.3 Ejemplo de cómo crear una vista con SQL 47
10. Gestión de la seguridad 50
10.1 Introducción 50
10.2 Usuarios 50
10.3 Roles 51
10.4 Privilegios 51
10.5 Perfiles 52
10.6 Diferencias entre roles y perfiles 52
11. Lenguaje de definición de datos (DML, Data Manipulation Languaje) 57
11.1 Concepto 57
11.2 Comandos DDL 57
11.3 Buenas prácticas y gestión de cambios de esquema (DDL) 60
12. Lenguaje de manipulación de datos (DCL, Data Control Language) 65
12.1 Concepto 65
12.2 Comandos DML 66
12.3 Ejemplo de comandos de manipulación de datos 68
13. Lenguaje de control de datos (DCL, Data Control Language) 72
13.1 Concepto 72
13.2 Comandos DCL 73
13.3 Ejemplo de comandos de control de datos 74
14. Resumen 77
Glosario 79
Bibliografía 81

Objetivos de la unidad

Los objetivos de aprendizaje de esta unidad son:

- Comprender los conceptos básicos de las bases de datos relacionales, incluyendo tablas, filas, columnas, claves primarias y secundarias, relaciones y normalización.
- Conocer los distintos sublenguajes que integran el lenguaje SQL (Structured Query Language) para definir y manipular datos.
- Comprender cómo se relacionan las tablas y cómo se utilizan las claves primarias y secundarias para establecer relaciones entre ellas.
- Conocer su estructura básica de seguridad.

Tech Hub <FP> euroformac

1. Introducción

Las **bases de datos relacionales** son un tipo de base de datos que utiliza un modelo de datos basado en relaciones para representar y almacenar datos.

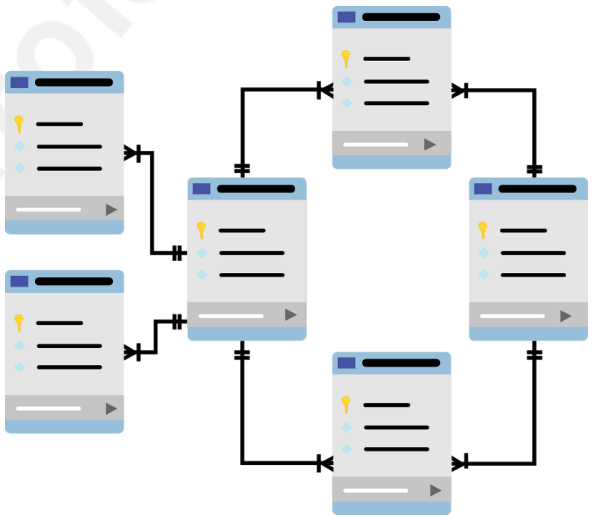
En una base de datos relacional, los datos se organizan en tablas, que contienen filas y columnas. Cada columna representa un tipo de información y cada fila representa un registro individual.

Sabías que

Las relaciones se establecen entre tablas utilizando claves primarias y secundarias. La clave primaria es una columna o conjunto de columnas que identifica de manera única a cada registro en una tabla, mientras que la clave secundaria hace referencia a una clave primaria en otra tabla.

El Lenguaje de Consulta Estructurado (SQL) es el lenguaje principal utilizado para interactuar con una base de datos relacional. Con SQL, se pueden realizar operaciones como crear y modificar tablas, agregar y eliminar datos, y realizar consultas complejas.

Las bases de datos relacionales son muy útiles para aplicaciones que requieren una gran cantidad de datos organizados y fácilmente accesibles, como sistemas de gestión de información empresarial, aplicaciones de e-commerce y aplicaciones de análisis de datos.



Así mismo, también presentan algunos desafíos, como la escalabilidad y la eficiencia en el manejo de datos no estructurados.

2. Modelo de datos

El modelo de datos relacional se basa en la **teoría matemática de las relaciones,** donde los datos se representan como un conjunto de entidades relacionadas entre sí. Cada entidad es representada por una tabla en la base de datos, y cada relación es representada por una clave primaria y secundaria.

Así mismo, permite una representación flexible y escalable de los datos, y es compatible con la mayoría de los sistemas de gestión de bases de datos relacionales.

Además, también permite la normalización de los datos, lo que ayuda a evitar redundancias y a mejorar la eficiencia de las consultas.

En conclusión, el modelo de datos es una herramienta fundamental para diseñar y administrar bases de datos relacionales eficientes y escalables.

Definición

Un **modelo de datos** en bases de datos relacionales es una representación abstracta de los datos y las relaciones entre ellos. Este modelo permite definir cómo se organizan y relacionan los datos en una base de datos relacional, y cómo se accede a ellos.

2.1 Principales características de los modelos de datos

Los **modelos de datos** se caracterizan por incluir y disponer de:

Característica	Descripción
Tablas	Los datos se organizan en tablas, donde cada tabla representa una entidad o un conjunto de datos relacionados.
Claves	Las claves primarias y secundarias son utilizadas para establecer relaciones entre tablas y garantizar la integridad de los datos.
Normalización	El modelo de datos relacional permite la normalización de los datos, lo que ayuda a evitar redundancias y a mejorar la eficiencia de las consultas.
Lenguaje de consulta	El lenguaje de consulta estructurado (SQL) es el lenguaje principal utilizado para interactuar con una base de datos relacional.
Flexibilidad	El modelo de datos relacional permite la representación flexible y escalable de los datos, y es compatible con la mayoría de los sistemas de gestión de bases de datos relacionales.
Integridad de los datos	El modelo de datos relacional garantiza la integridad de los datos mediante la implementación de restricciones y reglas de negocio.
Escalabilidad	El modelo de datos relacional es escalable, lo que permite añadir nuevas tablas y datos a la base de datos a medida que la aplicación crece.

2.2 Diseño de un Modelo de Datos

El diseño eficiente de un modelo de datos implica varios factores importantes, incluyendo:

Factor	Descripción
Normalización	La normalización del modelo de datos ayuda a evitar redundancias y a mejorar la eficiencia de las consultas. Es importante normalizar el modelo de datos hasta el grado adecuado, sin caer en la sobre-normalización.
Identificación de las entidades	Es importante identificar las entidades relevantes y las relaciones entre ellas, y representarlas adecuadamente en el modelo de datos.
Uso de claves	Las claves primarias y secundarias son fundamentales para establecer relaciones entre tablas y garantizar la integridad de los datos. Es importante identificar las claves adecuadas para cada tabla.
Optimización de las consultas	Es importante optimizar las consultas a la base de datos para garantizar un rendimiento óptimo. Esto incluye la creación de índices, la utilización de consultas eficientes y la optimización de la estructura de las tablas.
Escalabilidad	El modelo de datos debe ser escalable, lo que permite añadir nuevas tablas y datos a medida que la aplicación crece.
Flexibilidad	El modelo de datos debe ser flexible para permitir cambios en el futuro. Esto incluye la posibilidad de añadir nuevas tablas y campos, y de modificar la estructura existente.

Por tanto, un diseño eficiente implica una combinación de normalización, identificación de las entidades, uso de claves, optimización de las consultas, escalabilidad y flexibilidad. Un modelo de datos bien diseñado es fundamental para garantizar un rendimiento óptimo y una gestión eficiente de los datos.

Veamos estos contenidos aplicados a un ejemplo práctico – Diseño de un modelo de datos para una librería

En la "Librería El Saber", además de vender en tienda física, quieren empezar a registrar ventas por correo electrónico. Necesitan un sistema que almacene la información de los clientes, los libros y cada venta de forma ordenada, evitando repeticiones innecesarias y permitiendo consultas rápidas.

¿Cómo diseñar un modelo de datos relacional que cumpla estos requisitos, usando tablas, claves primarias y secundarias, y evitando redundancias?

Solución:

1. **Identificar entidades y sus datos básicos:**
   - Cliente: datos de la persona que compra.
   - Libro: información de cada título disponible.
   - Venta: registro de cada transacción.
2. **Definir tablas y campos:**

- Tabla Cliente
  - Campos:
    - IDCliente (clave primaria)
    - Nombre
    - Email
- Tabla Libro
  - Campos:
    - ISBN (clave primaria)
    - Título
    - Autor
    - Precio

- Tabla Venta
  - Campos:
    - IDVenta (clave primaria)
    - Fecha
    - IDCliente (clave secundaria que vincula a Cliente)
    - ISBN (clave secundaria que vincula a Libro)

3. **Uso de claves:**
   - Cada tabla tiene una clave primaria única (IDCliente, ISBN, IDVenta).
   - En Venta, las claves secundarias (IDCliente y ISBN) garantizan la relación con Cliente y Libro.
4. **Normalización y eliminación de redundancias:**
   - Los datos de cliente (nombre, email) se guardan sólo en la tabla Cliente.
   - Los datos de libro (título, autor, precio) se guardan sólo en la tabla Libro.
   - La tabla Venta sólo almacena las referencias necesarias (fecha y claves).
5. **Optimización de consultas:**
   - Para acelerar búsquedas de un cliente por email, se podría crear un índice en la columna Email de la tabla Cliente.
   - Para listar rápidamente todas las ventas de un libro concreto, se crearía un índice en ISBN dentro de Venta.
6. **Flexibilidad y escalabilidad:**
   - Si en el futuro se añade un "DetalleVenta" para soportar ventas de varios libros en una misma transacción, bastará con crear esa nueva tabla y vincularla mediante claves secundarias.
   - Para integrar proveedores, se puede añadir una tabla Proveedor con su propia clave primaria y relacionarla con Libro.

Esquema relacional resultante:

- Cliente(IDCliente PK, Nombre, Email)
- Libro(ISBN PK, Título, Autor, Precio)
- Venta(IDVenta PK, Fecha, IDCliente FK, ISBN FK)

Con este diseño el modelo aprovecha todas las características de los datos relacionales: tablas organizadas, claves para las relaciones, normalización para evitar duplicados, posibilidad de añadir índices para mejorar el rendimiento, y espacio para crecer con nuevas tablas sin romper la estructura original.

3. Terminología del Modelo Relacional

3.1 Relaciones, atributos, tuplas

Como dijimos anteriormente, el modelo relacional de bases de datos es un modelo matemático que define la estructura de una base de datos y cómo se relacionan los datos entre sí.

Aquí están algunos **términos clave de la terminología relacional:**

Término	Descripción
Relaciones	Una relación es un conjunto de pares ordenados (tuplas) que describen las relaciones entre las entidades en una base de datos. Cada relación está compuesta por un conjunto de atributos.
Atributos	Un atributo es una característica o propiedad de una entidad. Por ejemplo, el atributo "matrícula" podría ser un atributo de una entidad "vehículo".
Tuplas	Una tupla es una fila o registro en una tabla de una base de datos relacional. Una tupla representa una instancia de una entidad y contiene valores para cada uno de sus atributos.

Además de estos tres conceptos, existe un cuarto que también es fundamental en los modelos relacionales, y son las entidades o tablas que lo conforman:

Término	Descripción
Entidades o Tablas	Una entidad o tabla es una colección de tuplas que comparten una estructura común de atributos. Cada entidad o tabla representa una clase de objetos o un tipo de datos específico

Estos términos son la base de la terminología relacional y se utilizan para describir la estructura de una base de datos relacional y cómo se relacionan los datos entre sí.

A continuación, mostramos un ejemplo sobre qué representa cada uno de los conceptos explicados. En un sistema de gestión de flotas, la entidad denominada **Vehículos** agrupa en una única tabla toda la información relativa a los coches disponibles. Cada tabla en un modelo relacional representa una colección de datos homogéneos.

Dentro de la tabla **Vehículos** se definen tres atributos:

- **Matrícula**, que identifica de forma única cada coche.
- **Marca**, que especifica el fabricante.
- **Modelo**, que indica la denominación comercial del vehículo.

Cada fila de la tabla es una tupla, es decir, una instancia de la entidad con valores para cada atributo. Por ejemplo, una tupla podría almacenarse así:

1234-ABC|Toyota|Corolla

- Otra tupla podría ser:

5678-XYZ | Ford | Focus

Por último, la relación **Vehículos** se entiende como el conjunto de todas las tuplas almacenadas en dicha tabla. Matemáticamente, una relación asocia cada combinación de valores de atributos a una tupla concreta. En el ejemplo, la relación Vehículos está formada por las tuplas

(1234-ABC, Toyota, Corolla),

(5678-XYZ, Ford, Focus)

Este ejemplo impersonal muestra claramente que:

- La **entidad** o **tabla** es el contenedor de información homogénea.
- Los **atributos** son las columnas que describen las propiedades de cada registro.
- Las **tuplas** son las filas que recogen los valores de esos atributos para cada elemento.
- La **relación** es el conjunto total de tuplas que conforman la tabla.

A continuación, detallaremos un poco más las características que describen a cada uno de estos términos. Observarlas garantizará la eficiencia, la integridad, y la precisión de los datos en una base de datos relacional.

3.2 Principales características de las relaciones

Las características más importantes de las Relaciones son las siguientes:

Característica	Descripción
Cardinalidad	La cardinalidad describe la cantidad de tuplas que pueden estar relacionadas entre sí. La cardinalidad puede ser uno a uno, uno a muchos o muchos a muchos.
Identificadores únicos	Cada tupla en una relación debe tener un identificador único, como una clave primaria, para diferenciarla de otras tuplas en la misma relación.
Integridad de datos	Las relaciones deben cumplir ciertas reglas de integridad de datos, como la integridad referencial, para asegurar que los datos sean precisos y consistentes.
Normalización	Las relaciones deben estar normalizadas para evitar la redundancia de datos y mejorar la eficiencia y escalabilidad de la base de datos.
Reglas de integridad	Las relaciones pueden tener reglas de integridad que limitan los valores que pueden tener los atributos de las tuplas relacionadas.

3.3 Principales características de los Atributos

Las características de los Atributos son las siguientes:

Característica	Descripción
Identificadores únicos	Los atributos pueden ser utilizados como identificadores únicos para diferenciar las tuplas en una relación.
Tipo de datos	Cada atributo tiene un tipo de dato asociado, como número, texto o fecha, que define el tipo de valores que puede almacenar.
Valores nulos	Algunos atributos pueden permitir valores nulos, lo que significa que pueden estar vacíos o sin especificar.
Restricciones	Los atributos pueden tener restricciones, como valores mínimos o máximos, para asegurar la integridad de los datos.
Claves	Algunos atributos pueden ser claves, como claves primarias o claves externas, para establecer relaciones entre las tuplas en diferentes relaciones.

3.4 Principales características de las tuplas

Las características de las tuplas son las siguientes:

Característica	Descripción
Identificadores únicos	Cada tupla debe tener un identificador único, como una clave primaria, para diferenciarse de otras tuplas en la misma relación.
Atributos	Cada tupla está compuesta por un conjunto de atributos, que describen diferentes aspectos de la tupla.
Valores	Cada atributo de una tupla tiene un valor asociado que describe una característica específica de la tupla.
Integridad de datos	Deben cumplir ciertas reglas de integridad de datos, como la integridad referencial, para asegurar que los datos sean precisos y consistentes.
Reglas de integridad	Pueden tener reglas de integridad que limitan los valores que pueden tener los atributos de la tupla.

3.5 Principales características de las entidades o tablas

Las características de las entidades o tablas son las siguientes:

Característica	Descripción
Identificadores únicos	Cada entidad o tabla debe tener un identificador único, como una clave primaria, para diferenciarse de otras entidades o tablas en la misma base de datos.
Atributos	Cada entidad o tabla está compuesta por un conjunto de atributos, que describen diferentes aspectos de la entidad o tabla.
Tuplas, filas o registros	Cada entidad o tabla contiene una serie de filas o registros, que describen diferentes instancias u ocurrencias de la entidad o tabla.
Integridad de datos	Deben cumplir ciertas reglas de integridad de datos, como la integridad referencial, para asegurar que los datos sean precisos y consistentes.
Reglas de integridad	Pueden tener reglas de integridad que limitan los valores que pueden tener los atributos de la entidad o tabla.

4. Tipos de datos

4.1 Introducción a los tipos de datos

Al especificar un tipo de dato para una columna en una base de datos, estás limitando los valores que se pueden guardar en esa columna a aquellos que sean válidos para ese tipo de dato. Esto asegura que los datos sean coherentes y que las operaciones realizadas en ellos sean válidas.

Definición

Un **tipo de dato** es una categoría de valores que comparten ciertas características y se utilizan para describir los datos que se guardarán en una base de datos o en un lenguaje de programación. Por ejemplo, un tipo de dato "fecha" podría incluir valores que representen fechas y horas específicas. Un tipo de dato "entero" podría incluir valores que representen números enteros.

Los diferentes sistemas de base de datos y lenguajes de programación pueden tener diferentes tipos de datos disponibles, pero muchos de ellos comparten tipos de datos básicos como los siguientes:

Tipo	Descripción
Entero	Para almacenar números enteros, como el número de unidades vendidas.
Decimal	Para almacenar números con decimales, como los precios de los productos.
Cadena de caracteres	Para almacenar texto, como los nombres de las personas o las descripciones de los productos.
Fecha y hora	Para almacenar fechas y horas, como la fecha de nacimiento o la hora de un evento.
Booleano	Para almacenar valores verdaderos o falsos, como un indicador de si una orden ha sido completada o no.

Estos son algunos de los tipos de datos más comunes en las bases de datos relacionales, pero dependiendo del sistema de gestión de base de datos utilizado, puede haber otros tipos de datos disponibles.

4.2 Principales tipos de datos en MySQL

MySQL soporta los siguientes tipos de datos:

Números enteros	INT, TINYINT, SMALLINT, MEDIUMINT, BIGINT.
Números decimales	FLOAT, DOUBLE.
Números de punto flotante	FLOAT, DOUBLE.
Fecha y hora	DATE, DATETIME, TIME, YEAR, TIMESTAMP.
Cadena de caracteres	CHAR, VARCHAR, TINYTEXT, TEXT, MEDIUMTEXT, LONGTEXT.
Binarios	BLOB, TINYBLOB, MEDIUMBLOB, LONGBLOB.
Boooleanos	BOOLEAN, TINYINT(1).
Enumerados	: ENUM.

A continuación, se muestra para cada familia de tipos de datos en MySQL, un ejemplo de cuándo y por qué se utilizaría en un escenario real de desarrollo web:

1. **Enteros (INT, TINYINT, SMALLINT, MEDIUMINT, BIGINT)**

- **Cuándo usarlo:** para valores discretos y exactos que no necesiten decimales, como identificadores, contadores o estados codificados.
- **Ejemplos de uso habitual en MySQL:**
  - Campo usuario_id en una tabla de usuarios: se suele usar **INT** porque cubre de 4 a 4.000 millones de registros, ideal para la mayoría de sitios web.

- Campo edad en un perfil de usuario: **TINYINT** (rango -128 a 127) es suficiente para edades entre 0 y 120, y ahorra espacio.
- Campo visitas_mensuales para medir el número de visitas único al mes: **MEDIUMINT** (hasta ~16 millones) si el tráfico es muy alto.

2. **Decimales (FLOAT, DOUBLE)**

- **Cuándo usarlo:** para valores numéricos con parte fraccionaria donde no haga falta precisión absoluta.
- **Ejemplos habituales:**
  - Campo ratio_clicks en estadísticas de campaña publicitaria: un número entre 0 y 1 con dos decimales, donde la ligera imprecisión de **FLOAT** resulta aceptable y ahorra espacio.
  - Campo temperatura en una app meteorológica: **DOUBLE** si se requieren más cifras significativas y mayor rango.

3. **Fecha y hora (DATE, DATETIME, TIME, YEAR, TIMESTAMP)**

- **Cuándo usarlo:** para registrar momentos o periodos temporales.
- **Ejemplos habituales:**
  - Campo fecha_nacimiento en perfiles de usuario: **DATE**, porque sólo importa la fecha (día, mes, año).
  - Campo fecha_registro en un formulario de inscripción: **DATETIME** si se quiere guardar además la hora exacta.
  - Campo ultima_visita en una sesión web: **TIMESTAMP**, útil para operaciones automáticas de caducidad basadas en zona horaria del servidor.

4. **Cadenas de caracteres (CHAR, VARCHAR, TINYTEXT, TEXT, MEDIUMTEXT, LONGTEXT)**

- **Cuándo usarlo:** para información alfanumérica de longitud variable o fija.
- **Ejemplos habituales:**
  - Campo codigo_promocional con siempre 8 caracteres: **CHAR(8)**, ya que el tamaño es fijo y mejora ligeramente el rendimiento.
  - Campo email de usuario: **VARCHAR(100)**, longitud variable hasta 100 caracteres.

- Campo comentario_blog: **TEXT**, para permitir entradas de hasta 65 000 caracteres sin desperdiciar espacio si se usan campos más cortos.

5. **Binarios (BLOB, TINYBLOB, MEDIUMBLOB, LONGBLOB)**

- **Cuándo usarlo:** para almacenar datos no textuales (imágenes, archivos).
- **Ejemplos habituales en MySQL web:**
  - **TINYBLOB** para iconos o miniaturas (menos de 255 bytes).
  - **BLOB** para imágenes de perfil (hasta 65 000 bytes).
  - **LONGBLOB** si se guarda un documento PDF completo o vídeo en la propia base de datos (varios megabytes).

6. **Booleanos (BOOLEAN, TINYINT(1))**

- **Cuándo usarlo:** para valores verdadero/falso, sí/no.
- **Ejemplo habitual:**
  - Campo activo en la tabla de productos, donde 1 significa "disponible" y 0 "agotado". MySQL lo implementa internamente como **TINYINT(1)**.

7. **Enumerados (ENUM)**

- **Cuándo usarlo:** para campos con un conjunto pequeño y fijo de valores posibles.
- **Ejemplos habituales:**
  - Campo estado_pedido en una tienda online con valores "pendiente", "enviado" o "entregado". ENUM permite asegurar que sólo se usen esas tres opciones, evitando datos erróneos.

4.3 Principales tipos de datos en Oracle

Oracle soporta los siguientes tipos de datos:

Números enteros	INTEGER, NUMBER (con precisión y escala específicas).
Números decimales	NUMBER (con precisión y escala específicas).
Números de punto flotante	BINARY_FLOAT, BINARY_DOUBLE.
Fecha y hora	DATE, TIMESTAMP, TIMESTAMP WITH TIME ZONE, INTERVAL YEAR TO MONTH, INTERVAL DAY TO SECOND.
Cadena de caracteres	CHAR, VARCHAR2, NCHAR, NVARCHAR2, CLOB, NCLOB.
Binarios	BLOB, BFILE.
Boooleanos	BOOLEAN.
Tipos de datos específicos de Oracle	RAW, LONG RAW, ROWID, UROWID, NUMBER (con precisión y escala específicas).

En entornos que utilizan **Oracle** como **SGBD**, la elección del tipo de dato adecuado debe basarse en la naturaleza de la información y en los requisitos de precisión, rendimiento y compatibilidad.

A continuación, se describen casos prácticos de uso para cada familia de tipos de datos en Oracle:

1. Números enteros (INTEGER, NUMBER con precisión)

   Cuando se necesita un identificador o un contador sin decimales y con rango personalizable.

   - Ejemplo: la columna empleado_id puede declararse como INTEGER si no se prevén más de 2000 millones de registros.
   - Ejemplo: la columna num_pedido en una aplicación financiera puede usarse como NUMBER(10,0) para asegurar un rango hasta diez dígitos exactos.

2. Números decimales (NUMBER con precisión y escala)

   Para valores monetarios o cálculos donde la exactitud es imprescindible.

   - Ejemplo: en la tabla producto, precio_unitario se define como NUMBER(8,2) para dos decimales de céntimos.
   - Ejemplo: tipos de cambio en una tabla cotizaciones pueden almacenarse como NUMBER(12,6) si se requieren seis decimales de precisión.

3. Punto flotante (BINARY_FLOAT, BINARY_DOUBLE)

   En aplicaciones científicas o de ingeniería donde se tolera cierta imprecisión y se busca mayor velocidad.

   - Ejemplo: lecturas de sensores de temperatura en tiempo real pueden usarse con BINARY_FLOAT para reducir el uso de memoria.
   - Ejemplo: en simulaciones complejas de física financiera, se opta por BINARY_DOUBLE para un mayor rango dinámico.

4. Fecha y hora (DATE, TIMESTAMP, TIMESTAMP WITH TIME ZONE, INTERVAL…)

   Para almacenar fechas simples o instantes con zona horaria y duraciones.

   Ejemplo: columna fecha_nacimiento usa DATE ya que solo interesa el día.

   - Ejemplo: fecha_evento en una app global se define como TIMESTAMP WITH TIME ZONE para gestionar usuarios en diferentes husos.
   - Ejemplo: duracion_evento puede usarse como INTERVAL HOUR TO SECOND si se quiere calcular lapsos exactos.

5. Cadenas de caracteres (CHAR, VARCHAR2, NCHAR, NVARCHAR2, CLOB, NCLOB)

   - Según longitud, necesidad de caracteres Unicode o volúmenes de texto. Ejemplo: pais como CHAR(2) para dos letras ISO.
   - Ejemplo: email_usuario como VARCHAR2(100) por su longitud variable.
   - Ejemplo: descripcion_producto en un catálogo extenso puede usarse como CLOB para textos de varios miles de caracteres.

- Ejemplo: comentario_multilingüe se define como NVARCHAR2(200) para soportar caracteres no latinos.

6. Binarios (BLOB, BFILE)

   Para almacenar imágenes, documentos o archivos externos.

   - Ejemplo: foto_perfil en la tabla usuarios como BLOB para guardar JPEG directamente en la base de datos.
   - Ejemplo: manual_pdf se puede referenciar con BFILE cuando el PDF se guarda en el sistema de ficheros y solo se necesita un puntero.

7. Booleanos (BOOLEAN)

   Para valores verdadero/falso en lógica de aplicaciones PL/SQL.

   - Ejemplo: columna activo en clientes puede definirse como BOOLEAN para indicar estado de registro, usado internamente en procedimientos almacenados.

8. Tipos específicos de Oracle (RAW, LONG RAW, ROWID, UROWID)

   Cuando se requieren datos binarios crudos o referencias internas:

   - RAW(16) para UUIDs.
   - ROWID para almacenar direcciones físicas de fila y acelerar consultas de mantenimiento.

Con estas pautas, cada tipo de dato en Oracle se elige de acuerdo al contexto de la aplicación: si se busca máxima precisión (NUMBER), compatibilidad internacional (TIMESTAMP WITH TIME ZONE, NVARCHAR2), eficiencia en cálculos (BINARY_FLOAT) o gestión de grandes volúmenes de texto o ficheros (CLOB, BFILE).

4.4 Principales tipos de datos en SQL Server:

SQL Server soporta los siguientes tipos de datos:

Números enteros	TINYINT, SMALLINT, INT, BIGINT
Números decimales	DECIMAL, NUMERIC
Números de punto flotante	FLOAT
Fecha y hora	DATE, TIME, DATETIME, DATETIME2, SMALLDATETIME, DATETIMEOFFSET
Cadena de caracteres	CHAR, VARCHAR, TEXT, NCHAR, NVARCHAR, NTEXT
Binarios	BINARY, VARBINARY, IMAGE
Boooleanos	BIT
Tipos de datos específicos de SQL Server	UNIQUEIDENTIFIER, ROWVERSION (timestamp)

La selección del tipo de dato en SQL Server depende de la naturaleza de la información, el volumen esperado y los requisitos de rendimiento. A continuación, se describen, de forma práctica e impersonal, ejemplos de cuándo es conveniente elegir cada familia de tipos de datos:

Enteros (TINYINT, SMALLINT, INT, BIGINT). Cuando se almacenan contadores o identificadores sin decimales:

- Un campo IdUsuario suele declararse como INT, ya que cubre hasta unos 2 000 millones de registros, suficiente para la mayoría de aplicaciones web.
- Para un indicador de estado con pocos valores (por ejemplo, niveles del 0 al 255), TINYINT permite ahorrar espacio.
- En tablas de archivos muy voluminosos (logs con decenas de millones de filas), BIGINT evita desbordamientos de contador.

Decimales (DECIMAL, NUMERIC). Para datos monetarios o valores donde la precisión exacta es crítica:

- En una tabla de Productos, PrecioUnitario podría definirse como DECIMAL(10,2) para garantizar dos decimales de céntimos sin errores de redondeo.
- En facturación a multinivel, descuentos o estadísticas financieras que exijan mayor precisión, NUMERIC(15,6) asegura hasta seis decimales.

Punto flotante (FLOAT). En escenarios científicos o de análisis estadístico donde se tolera ligera imprecisión a cambio de mayor rango y velocidad:

- Mediciones de sensores industriales que produzcan valores muy grandes o muy pequeños pueden usar FLOAT para optimizar espacio.
- Cálculos de posicionamiento geográfico (latitud/longitud) en aplicaciones de mapas, donde el margen de error es aceptable.

Fecha y hora (DATE, TIME, DATETIME, DATETIME2, SMALLDATETIME, DATETIMEOFFSET). Según el nivel de detalle y la necesidad de husos horarios:

- DATE es apropiado para fechas de nacimiento o aniversarios, donde no importa la hora.
- DATETIME2 ofrece mayor precisión de fracciones de segundo que DATETIME, útil en sistemas de trading con alta frecuencia.
- DATETIMEOFFSET incorpora zona horaria, ideal para registros de eventos en aplicaciones globales.
- SMALLDATETIME (precisión de un minuto) puede bastar en sistemas de reserva de salas o turnos, con almacenamiento más compacto.

Cadenas de caracteres (CHAR, VARCHAR, TEXT, NCHAR, NVARCHAR, NTEXT). Según la longitud esperada y necesidad de Unicode:

- CHAR(2) para códigos de país ISO, de longitud fija.
- VARCHAR(100) para direcciones de correo, variable y con límite superior razonable.
- NVARCHAR(200) en aplicaciones multilíngües que requieran caracteres Unicode.

- TEXT o NTEXT (obsoletos en versiones modernas) en escenarios heredados de almacenamiento de textos muy extensos (blogs antiguos, descripciones).

Binarios (BINARY, VARBINARY, IMAGE). Para datos binarios como imágenes, archivos o valores generados por la aplicación:

- VARBINARY(256) para almacenar pequeños hashes o tokens.
- IMAGE (obsoleto en versiones recientes) en sistemas legados que guardan fotografías de perfil o documentos escaneados directamente en la base de datos.

Booleanos (BIT). Cuando se registran banderas de verdadero/falso de forma compacta:

- Un campo IsActivo en la tabla Usuarios utiliza BIT para indicar estado, ocupando solo un bit por registro.

Tipos específicos (UNIQUEIDENTIFIER, ROWVERSION). Para valores especiales de SQL Server:

- UNIQUEIDENTIFIER almacena GUIDs, perfectos para claves primarias distribuidas o sincronización entre bases de datos.
- ROWVERSION (alias timestamp) genera un número único en cada modificación de fila, muy útil para detectar cambios concurrentes en sistemas de replicación o control de acceso optimista.

Con estos ejemplos se aprecia cómo, dependiendo de si se buscan exactitud absoluta (DECIMAL), flexibilidad de zona horaria (DATETIMEOFFSET), optimización de espacio (TINYINT, BIT) o compatibilidad con GUIDs y concurrencia (UNIQUEIDENTIFIER, ROWVERSION), se selecciona el tipo de dato más adecuado en SQL Server.

4.5 Principales tipos de datos en DB2

DB2 soporta los siguientes tipos de datos:

Números enteros	SMALLINT, INTEGER, BIGINT
Números decimales	DECIMAL, NUMERIC
Números de punto flotante	REAL, DOUBLE
Fecha y hora	DATE, TIME, TIMESTAMP
Cadena de caracteres	CHAR, VARCHAR, CLOB
Binarios	BLOB
Boooleanos	BOOLEAN
Tipos de datos específicos de DB2	GRAPHIC, VARGRAPHIC, DBCLOB

En entornos que emplean IBM DB2, la selección del tipo de dato adecuado busca equilibrar precisión, rendimiento y uso de espacio según la naturaleza de la información. A continuación, se presentan ejemplos prácticos de cuándo conviene elegir cada familia de tipos de datos:

Enteros (SMALLINT, INTEGER, BIGINT). Para valores discretos y exactos sin decimales, como identificadores y contadores.

- En una base de datos de facturación interna, el campo cliente_id suele definirse como **INTEGER**, porque cubre hasta unos 2000 millones de registros.
- En un histórico de visitas diarias donde el recuento nunca supera 32 767, **SMALLINT** ahorra espacio.
- Para tablas de logs con más de 2000 millones de registros acumulados, **BIGINT** evita desbordamientos.

Decimales (DECIMAL, NUMERIC). Cuando se manejan valores monetarios o cálculos que requieren exactitud absoluta.

- La columna precio_unitario de un catálogo de productos se declara como **DECIMAL(9,2)** para dos decimales de céntimos sin errores de redondeo.

- Para porcentajes de descuento que pueden tener cuatro decimales, **NUMERIC(5,4)** ofrece el rango y la precisión necesarios.

Punto flotante (REAL, DOUBLE). Al igual que en otros SGBD, en aplicaciones científicas o de simulación donde se tolera pequeña imprecisión a cambio de rango y velocidad.

- Para lecturas de sensores de temperatura ambiental en tiempo real, **REAL** proporciona suficiente precisión con uso de memoria reducido.
- En modelos de predicción financiera que manejan valores muy grandes o muy pequeños, **DOUBLE** garantiza un rango dinámico mayor.

Fecha y hora (DATE, TIME, TIMESTAMP). Para captura de fechas, horas y marcas temporales con distintos niveles de detalle.

- El campo fecha_contrato en recursos humanos usa **DATE**, pues solo importa el día en que se firmó.
- La columna hora_turno en una agenda de turnos se define como **TIME** para almacenar solo la hora de inicio.
- Para auditoría de cambios, fecha_modificacion se declara **TIMESTAMP**, registrando día, hora y fracciones de segundo.

Cadenas de caracteres (CHAR, VARCHAR, CLOB). Según longitud fija o variable y volúmenes de texto.

- En un sistema de códigos de país ISO, pais_codigo se guarda como **CHAR(2)**, valor fijo y compacto.
- Para nombres de usuario de longitud variable hasta 50 caracteres, **VARCHAR(50)** evita desperdiciar espacio.
- En un portal de noticias, los artículos completos se almacenan como **CLOB**, permitiendo textos de varios megabytes.

Binarios (BLOB). Para almacenar objetos binarios grandes, como imágenes o documentos.

- Una aplicación de gestión documental guarda cada PDF escaneado en un campo **BLOB**, facilitando su recuperación íntegra.

Booleanos (BOOLEAN). Para banderas verdadero/falso en lógicas de aplicación.

- El campo activo en la tabla de promociones usa **BOOLEAN** para indicar si una promoción está vigente.

Tipos específicos de DB2 (GRAPHIC, VARGRAPHIC, DBCLOB). Para datos multibyte y colecciones de texto Unicode o multilenguaje.

- El nombre de cliente en una base multilingüe se define como **VARGRAPHIC(100)** para soportar caracteres de idiomas con caracteres especiales.
- Comentarios extensos en varios idiomas se guardan en **DBCLOB**, asegurando que todo el texto se almacena correctamente sin truncar.

Con estos ejemplos, queda claro cómo DB2 ofrece un catálogo de tipos que cubre desde identificadores simples y valores monetarios exactos hasta grandes volúmenes de texto y datos multilingües, permitiendo elegir siempre el tipo que mejor se adapta al uso real de la aplicación.

5. Claves primarias

5.1 Concepto de clave primaria

Las claves primarias pueden ser una sola columna, como un número de identificación, o puede ser un conjunto de columnas que juntas proporcionan una identificación única. Por ejemplo, podría haber una tabla de productos con una clave primaria que consista en las columnas "categoría" y "referencia".

Definición

Una **clave primaria** es una columna o conjunto de columnas en una tabla de base de datos que identifica de manera única a cada registro o fila en la tabla. La clave primaria se utiliza para establecer una relación entre tablas y asegurarse de que cada registro tenga una identificación única.

Establecer **una clave primaria** es una práctica importante en la creación de una base de datos relacional, ya que permite la integridad de los datos y facilita la gestión de las relaciones entre tablas. La clave primaria también se utiliza en la creación de índices, que ayudan a optimizar la consulta y el rendimiento de la base de datos.

Sabías que

Una **clave primaria autonumérica** es un tipo de clave primaria que se asigna automáticamente a cada nuevo registro en una tabla. Esta clave se genera mediante un algoritmo que incrementa el número de la última clave primaria utilizada en la tabla.

Se trata de una característica común en la mayoría de los sistemas de bases de datos y se utiliza para proporcionar una identificación única para cada registro en la tabla.

Además, como la clave se asigna automáticamente, se elimina la necesidad de que el usuario asigne manualmente una clave para cada registro, lo que reduce la posibilidad de errores y asegura la integridad de los datos.

5.2 Claves primarias simples vs claves primarias compuestas

Las claves primarias pueden ser **simples** o **compuestas**.

Característica	Descripción
Clave primaria simple	Una clave primaria simple es una columna única en una tabla que se utiliza para identificar de manera única a cada registro. Por ejemplo, una clave primaria simple en una tabla de clientes podría ser el número de identificación del cliente.
Clave primaria compuesta	Una clave primaria compuesta es una combinación de dos o más columnas en una tabla que se utiliza para identificar de manera única a cada registro. Por ejemplo, una clave primaria compuesta en una tabla de pedidos podría ser la combinación del número de identificación del cliente y la fecha del pedido.

La elección entre una clave primaria simple o compuesta depende de la naturaleza de los datos y del propósito de la base de datos. En algunos casos, una clave primaria simple puede ser suficiente para identificar de manera única a cada registro, mientras que, en otros casos, puede ser necesario utilizar una clave primaria compuesta para garantizar una identificación única de los registros.

Importante

Es importante tener en cuenta que una tabla solo puede tener una clave primaria, por lo que es importante elegir cuidadosamente el tipo de clave primaria que se utilizará. La clave primaria debe ser única, no nula y no cambiante para asegurar la integridad de los datos en la base de datos.

5.3 Ejemplos de claves primarias

Estos son algunos ejemplos de claves primarias:

Ejemplo	Descripción
Tabla de productos	Una clave primaria en esta tabla podría ser un número de identificación único para cada producto, como un número SKU (Stock Keeping Unit).
Tabla de clientes	Una clave primaria en esta tabla podría ser un número de identificación único para cada cliente, como un número de cliente autonumérico, o el CIF.
Tabla de pedidos	Una clave primaria en esta tabla podría ser un número de identificación único para cada pedido, como un número de orden autonumérico.
Tabla de empleados	Una clave primaria en esta tabla podría ser un número de identificación único para cada empleado, como un número de empleado autonumérico o su NIF/NIE.
Tabla de Vehículos	Una clave primaria en esta tabla podría ser un número de identificación único autonumérico o el código de matrícula.

La clave primaria depende del tipo de información que se está almacenando en la tabla y de cómo se utilizarán los datos en la base de datos. La idea principal es que la clave primaria identifique de manera única a cada registro en la tabla.

Veamos en el siguiente ejemplo práctico todos estos conceptos – Elección de claves primarias en un estudio de tatuajes

InkArt Studio es un pequeño estudio de tatuajes que quiere informatizar su gestión. Necesita almacenar información sobre los clientes que solicitan diseños, los tatuadores que los realizan, las citas programadas y los detalles de cada tatuaje. Se plantea diseñar cuatro tablas básicas: Clientes, Artistas, Citas y DetalleCita.

Para cada una de estas tablas, ¿qué tipo de clave primaria conviene usar —simple o compuesta— y por qué?

Solución

1. Tabla **Clientes**
   - Campos: ClienteID, Nombre, Teléfono, Email
   - **Clave primaria recomendada**: ClienteID
   - **Tipo de clave**: simple, autonumérica
   - **Justificación**: Cada cliente recibe un identificador único al registrarse. Un entero autoincremental garantiza unicidad, es fácil de gestionar y no cambia.
2. Tabla **Artistas**
   - Campos: ArtistaID, Nombre, Especialidad
   - **Clave primaria recomendada**: ArtistaID
   - **Tipo de clave:** simple, autonumérica
   - **Justificación:** Los tatuadores son pocos y con baja rotación. Un ID interno (por ejemplo, 1, 2, 3…) identifica sin ambigüedad a cada artista y sirve para relacionar con las citas.
3. Tabla **Citas**
   - Campos: ArtistaID, FechaHora, ClienteID, Estado
   - **Clave primaria recomendada:** (ArtistaID, FechaHora)
   - **Tipo de clave:** compuesta
   - **Justificación:**
   - **Regla de negocio:** un mismo tatuador no puede atender dos clientes a la vez.
   - La combinación de ArtistaID + FechaHora garantiza que no existan dos citas solapadas para el mismo artista.
   - Evita la necesidad de llevar un campo extra de ID de cita y, al ser inmutable, refuerza la integridad de horario.
4. Tabla **DetalleCita**
   - Campos: ArtistaID, FechaHora, DiseñoID, ZonaCuerpo, Duración
   - **Clave primaria recomendada:** (ArtistaID, FechaHora, DiseñoID)
   - **Tipo de clave:** compuesta

Justificación:

- Cada cita puede incluir varios diseños (por ejemplo, un tatuaje de flor y otro de calavera).
- Se utiliza la misma clave compuesta de Citas (ArtistaID+FechaHora) junto con DiseñoID para diferenciar cada diseño dentro de la cita.
- Garantiza que no se repita el mismo diseño dos veces en una misma cita, y permite gestionar cómodamente varios servicios por sesión.

Con este diseño:

- Las tablas que representan entidades independientes (Clientes, Artistas) usan **claves simples** para facilitar el registro y la referencia.
- Las tablas que aplican reglas de negocio sobre combinaciones de datos (Citas, DetalleCita) usan **claves compuestas** basadas en campos con significado, asegurando integridad y evitando duplicados inapropiados.

6. Índices

6.1 Concepto de índice

Los **índices** son **estructuras de datos utilizadas en bases de datos relacionales para mejorar el rendimiento de las consultas**. Estos índices se crean en una o más columnas de una tabla y permiten a los motores de base de datos realizar búsquedas más rápidas y eficientes.

6.2 Tipos de índices

Hay varios tipos de índices en bases de datos relacionales, analizándolos desde distintas perspectivas.

Desde un punto de vista de diseño, encontramos:

Tipo	Descripción
Índice simple	Un índice simple es un índice que se crea en una sola columna de una tabla. Este tipo de índice es útil para búsquedas precisas y para optimizar el rendimiento de consultas que utilizan igualdades o desigualdades.
Índice compuesto	Un índice compuesto es un índice que se crea en más de una columna de una tabla. Este tipo de índice es útil para optimizar el rendimiento de consultas que utilizan varias condiciones en diferentes columnas.

Desde un **punto de vista funcional**, podemos destacar:

Tipo	Descripción
Índice árbol B	Un índice árbol B es un tipo de índice que utiliza un árbol binario para organizar los datos. Este tipo de índice es eficiente para búsquedas y actualizaciones y es el tipo de índice más común en bases de datos relacionales.
Índice hash	Un índice hash es un tipo de índice que utiliza una función hash para mapear los datos a una ubicación en la tabla de índice. Este tipo de índice es eficiente para búsquedas y consultas en las que se utiliza una igualdad, pero no es eficiente para consultas que utilizan desigualdades.

Los índices son una herramienta valiosa para optimizar el rendimiento de las consultas en una base de datos relacional. Al elegir cuidadosamente los índices adecuados para una tabla, se puede mejorar significativamente el rendimiento de las consultas y el acceso a los datos en la base de datos.

Sabías que

El **índice árbol B** se organiza en una estructura de árbol binario, en la que cada nodo del árbol contiene una clave y un puntero a otro nodo o a una fila en la tabla.

Estos índices son muy eficientes para búsquedas y actualizaciones, ya que permiten a los motores de base de datos acceder a los datos de manera rápida y eficiente. Además, son muy flexibles y se pueden utilizar en una amplia variedad de situaciones, incluyendo búsquedas basadas en igualdades, desigualdades y rango.

Por ejemplo, si una tabla tiene una columna "apellido1" y se desea realizar una búsqueda rápida para encontrar todas las filas con un primer apellido específico, se puede crear un índice árbol B en la columna "apellido1". El motor de base de datos utilizará entonces el índice para acceder a los datos de manera rápida y eficiente.

El índice hash se utiliza para mejorar el rendimiento de las consultas. Este índice se basa en una función hash, que asigna a cada clave de la tabla un valor único, llamado "hasha".

Cuando se realiza una consulta que busca una fila específica en la tabla, el motor de base de datos utiliza el índice hash para calcular el valor hash para la clave buscada y luego buscar la fila correspondiente en la tabla. Este proceso es muy rápido y eficiente, lo que lo hace ideal para búsquedas de claves únicas.

Sin embargo, los índices hash no son adecuados para búsquedas basadas en rango o desigualdades, ya que no permiten una búsqueda secuencial. En estos casos, se utilizan otros tipos de índices, como los índices árbol B.

6.3 Características de los índices

Las **características** más importantes de los **índices en bases de datos relacionales** son:

Características	Descripción
Mejora del rendimiento	Los índices permiten a los motores de base de datos acceder a los datos con gran eficiencia, lo que mejora significativamente el rendimiento de las consultas.
Optimización de búsquedas	Los índices se utilizan para optimizar las búsquedas basadas en claves únicas o en rangos, lo que permite a los motores de base de datos encontrar los datos antes.
Ordenación de datos	Los índices pueden ordenar los datos de manera automática, lo que permite una búsqueda más eficiente.
Espacio adicional	Los índices requieren espacio adicional en el disco para almacenar su estructura y los valores de clave, por lo que deben utilizarse con moderación.
Mantenimiento	Los índices requieren mantenimiento periódico, especialmente si los datos en la tabla cambian con frecuencia.
Actualizaciones	Las actualizaciones a las tablas con índices pueden ser más lentas, ya que los índices deben actualizarse en consecuencia.

Los índices son una herramienta valiosa para optimizar el rendimiento de las consultas en una base de datos relacional, pero deben utilizarse con cuidado para evitar problemas de espacio y rendimiento.

Caso práctico resuelto – Optimización de consultas con índices en InkArt Studio

InkArt Studio ha crecido y ahora gestiona cientos de clientes, artistas y citas cada mes. Al buscar rápidamente todos los datos de un cliente, consultar el calendario de un tatuador o averiguar qué diseños se han usado con mayor frecuencia, las consultas a la base de datos empiezan a tardar demasiado.

¿Qué índices conviene crear en las tablas Clientes, Citas y DetalleCita para acelerar estas búsquedas, y qué tipo de índice (simple vs compuesto, B-tree vs hash) es el más adecuado en cada caso?

Solución:

1. **Búsqueda de cliente por correo**
   - **Tabla:** Clientes
   - **Columna a indexar:** Email
   - **Tipo de índice:** simple, hash (para igualdad)
   - **Justificación:** La mayoría de las búsquedas de cliente se hacen introduciendo exactamente su dirección de correo. Un índice hash sobre Email permite localizar al cliente de forma muy rápida cuando sólo importa la coincidencia exacta.
2. **Listado de citas de un artista ordenadas por fecha**
   - **Tabla:** Citas
   - **Columnas a indexar:** (ArtistaID, FechaHora)
   - **Tipo de índice:** compuesto, B-tree
   - **Justificación:** Frecuentemente se consulta "todas las citas de este tatuador, ordenadas por fecha". Un índice B-tree compuesto que cubra primero ArtistaID (para filtrar por artista) y luego FechaHora (para ordenar y permitir búsquedas por rango) acelera drásticamente estas consultas.

3. **Frecuencia de uso de cada diseño**
   - **Tabla:** DetalleCita
   - **Columnas a indexar:** DiseñoID
   - **Tipo de índice:** simple, B-tree
   - **Justificación:** Para generar estadísticas ("¿qué diseños son los más populares?"), se agrupan las filas por DiseñoID. Un índice B-tree en DiseñoID optimiza tanto la búsqueda de todas las tuplas con ese ID como las operaciones de agrupamiento y conteo.
4. **Detección de citas duplicadas**
   - **Tabla:** Citas
   - **Columnas a indexar:** (ArtistaID, FechaHora) como clave primaria
   - **Tipo de índice:** compuesto, B-tree
   - **Justificación:** Al usar esta combinación como clave primaria, además de impedir duplicados, se obtiene un índice que garantiza la integridad y acelera también la validación de nuevas citas para evitar solapamientos.

Con estos tres nuevos índices se consigue:

- Búsquedas instantáneas de cliente por correo (hash).
- Recorridos ordenados y por rangos en el calendario de artistas (B-tree compuesto).
- Cálculo rápido de estadísticas de uso de diseños (B-tree simple).

Esta estrategia mejora el rendimiento general del sistema sin crear demasiados índices, equilibrando espacio adicional y mantenimiento con la agilidad en las consultas más habituales.

7. El valor NULL

7.1 Concepto de valor NULL

Cuando se consulta una tabla, el motor de base de datos tratará los valores NULL como un valor especial y los diferenciará de los valores reales. Por ejemplo, las comparaciones con NULL pueden ser difíciles de manejar, ya que NULL no es igual a ningún otro valor, incluido otro NULL.

Definición

El **valor NULL** es un valor especial que indica que una columna en una tabla no tiene un valor asignado. Es importante tener en cuenta que NULL no es lo mismo que una cadena vacía o un cero, ya que NULL indica que el valor de la columna no está disponible o no es aplicable.

Es importante tener en cuenta que algunos motores de base de datos pueden proporcionar opciones para controlar cómo se manejan los valores NULL en las consultas y cómo se muestran los resultados, por lo que debe manejarse de manera cuidadosa en las consultas de la base de datos para evitar resultados no deseados.

7.2 Ejemplos de valores NULL

Algunos ejemplos de valores NULL en una base de datos relacional pueden incluir:

Ejemplos
Una columna que almacena la fecha de defunción de una persona, donde el valor NULL indica que no puede informarse porque aún no ha fallecido.
Una columna que almacena el número de teléfono de una persona, donde el valor NULL indica que la persona no proporcionó su número de teléfono.
Una columna que almacena el código del supervisor de un empleado, donde el valor NULL indica que el empleado no tiene un supervisor asignado.

Una columna que almacena la nota final de una asignatura obtenida por el alumno, donde el valor NULL indica que aún no está disponible. Sería ilógico poner un cero, porque no está suspendido aún, simplemente aún no se ha evaluado.

Estos son solo algunos ejemplos de cómo el valor NULL puede utilizarse en una base de datos relacional. La presencia de valores NULL en una tabla no necesariamente significa que la información sea incompleta o incorrecta, simplemente indica que no está disponible o no es aplicable.

Tech Hub <FP> euroformac

8. Claves foráneas

8.1 Concepto de calve foránea

La clave foránea se define en una tabla hija y hace referencia a una clave primaria en una tabla padre. La clave foránea garantiza que los valores en la tabla hija correspondan a valores válidos en la tabla padre.

Definición

Una **clave foránea** es un tipo de restricción en una base de datos relacional que establece una relación entre dos tablas, donde la clave foránea de una tabla hace referencia a una clave primaria en otra tabla, garantizando la integridad referencial entre las tablas.

Por ejemplo, imaginemos que tenemos una tabla de empleados y una tabla de departamentos. La tabla de empleados puede tener una clave foránea que haga referencia a la clave primaria de la tabla de departamentos. La clave foránea garantiza que cada empleado esté asignado a un departamento válido y existente.

Las claves foráneas son una herramienta importante para garantizar la integridad de los datos en una base de datos relacional y para evitar la entrada de datos inválidos o inconsistentes.

Además, también pueden mejorar la eficiencia y la claridad de las consultas, permitiendo la realización de operaciones de unión y consultas de relación.

8.2 Características de las claves foráneas

Las claves foráneas son características de las tablas en una base de datos relacional que definen una relación entre dos tablas. Algunas de sus características son:

Características	Descripción
Referencialidad	La clave foránea se refiere a una clave primaria en otra tabla, estableciendo una relación entre ellas.
Integridad referencial	Las claves foráneas ayudan a mantener la integridad de los datos al evitar la inserción de registros inválidos en la tabla que contiene la clave foránea.
Unidireccionalidad o bidireccionalidad	La relación puede ser unidireccional o bidireccional, dependiendo de si la tabla que contiene la clave foránea solo puede acceder a la otra tabla o ambas tablas pueden acceder a la información en ambas direcciones.
Acción en cascada	Se puede especificar cómo se deben actualizar o eliminar los registros relacionados cuando se realiza una operación en la tabla principal.
Restricciones	Se pueden aplicar restricciones a las claves foráneas para garantizar la integridad de los datos y evitar la inserción de valores inválidos.

8.3 Ejemplos de claves foráneas

Las claves foráneas se pueden usar en muchas situaciones en las que es necesario establecer relaciones entre tablas en una base de datos.

Estos son algunos ejemplos:

- Relación entre una tabla de clientes y una tabla de pedidos: la clave foránea en la tabla de pedidos se refiere a la clave primaria en la tabla de clientes, lo que indica que cada pedido está asociado a un cliente específico.
- Relación entre una tabla de productos y una tabla de proveedores: la clave foránea en la tabla de productos se refiere a la clave primaria en

la tabla de proveedores, lo que indica que cada producto es suministrado por un proveedor específico.

- Relación entre una tabla de departamentos y una tabla de empleados: la clave foránea en la tabla de empleados se refiere a la clave primaria en la tabla de departamentos, lo que indica que cada empleado pertenece a un departamento específico.

En el siguiente ejemplo práctico se abordan estos conceptos – Definición de claves foráneas en InkArt Studio

En InkArt Studio, además de las tablas Clientes, Artistas y Citas, existe una tabla **DetalleCita** que almacena qué diseños se realizaron en cada sesión. Para garantizar que no aparezcan citas o detalles que no correspondan a ningún cliente, artista o diseño, es necesario definir correctamente las claves foráneas y sus acciones de actualización o borrado.

¿Qué restricciones de clave foránea deben crearse en las tablas **Citas** y **DetalleCita**, y qué acciones (RESTRICT, CASCADE, SET NULL) conviene aplicar para mantener la coherencia al modificar o eliminar registros relacionados?

Solución

1. Tabla Citas

- **ClienteID → Clientes(ClienteID)**
  - Acción ON DELETE RESTRICT: impide borrar un cliente si todavía tiene citas agendadas, evitando citas "huérfanas".
  - Acción ON UPDATE CASCADE: si por algún motivo cambia el ID de un cliente, la cita se actualiza automáticamente.
- **ArtistaID → Artistas(ArtistaID)**
  - Acción ON DELETE RESTRICT: no se puede eliminar un artista que aún tenga citas registradas.
  - Acción ON UPDATE CASCADE: si se actualiza el identificador del artista, las citas mantienen la referencia correcta.

2. Tabla DetalleCita

- Clave compuesta (ArtistaID, FechaHora) → Citas(ArtistaID, FechaHora)

  - Acción ON DELETE CASCADE: al borrar una cita, se eliminan automáticamente sus detalles (los diseños asociados).
  - Acción ON UPDATE CASCADE: si cambian los datos de la cita (por ejemplo, se reprograma la hora), el detalle sigue referenciando la cita correcta.
- **DiseñoID → Diseños(DiseñoID)**
  - Acción ON DELETE RESTRICT: no se puede eliminar un diseño si ya ha sido utilizado en alguna cita.
  - Acción ON UPDATE CASCADE: si cambia la clave del diseño, los detalles la reflejan de inmediato.

De este modo, la base de datos:

- **Mantiene la integridad referencial**: no existen citas sin cliente o artista, ni detalles sin cita o diseño.
- **Controla eliminaciones**: un cliente o diseño no puede borrarse accidentalmente mientras esté en uso.
- **Automatiza limpiezas**: al anular una cita, sus detalles desaparecen, evitando registros obsoletos.

9. Vistas

9.1 Concepto de vista

Las vistas de bases de datos son una representación virtual de una o más tablas de una base de datos.

Una vista es esencialmente una capa de abstracción que encapsula una consulta SQL compleja bajo un nombre sencillo, de modo que los usuarios y las aplicaciones pueden tratarla como si fuera una tabla más, sin necesidad de conocer los detalles de su definición interna. Al no almacenar datos propios, cada acceso a la vista provoca que el sistema ejecute la sentencia subyacente y genere el conjunto de resultados al vuelo, garantizando que siempre refleje el estado más reciente de las tablas base.

Gracias a esta característica, las vistas permiten ocultar la complejidad de múltiples JOIN, WHERE y GROUP BY, ofreciendo un punto de entrada único para obtener datos ya combinados y filtrados según los requisitos del negocio. Además, facilitan la adaptación a futuros cambios en el esquema: si se modifican columnas o relaciones en las tablas originales, bastará con actualizar la definición de la vista para que el resto de consultas que la utilicen continúen funcionando sin alteraciones.

Existen tanto vistas simples (basadas en una única tabla) como vistas complejas (que incluyen varias tablas, agregaciones o subconsulta), y en muchos sistemas gestores pueden declararse como "updatable", de manera que algunas operaciones de inserción, actualización o borrado sobre la vista se traduzcan automáticamente en cambios válidos en las tablas subyacentes. De esta forma, las vistas no solo simplifican la lógica de acceso y refuerzan la seguridad —al exponer únicamente las columnas autorizadas—, sino que también pueden servir como punto de partida para vistas materializadas, que almacenen físicamente resultados precomputados para acelerar consultas pesadas o agregaciones frecuentes.

9.2 Características de las vistas

Algunas de las características son:

Características	Descripción
Representación virtual	Una vista es una representación virtual de una o más tablas, no almacena datos reales.
Fácil acceso	Permiten acceder a los datos de una o más tablas de una manera más fácil y organizada.

Filtrado de dato	Permiten filtrar y ordenar los datos en una tabla o combinación de tablas, permitiendo ver solo los datos relevantes.
Seguridad	Pueden ser utilizadas para restringir el acceso a ciertos datos o a ciertas operaciones en las tablas subyacentes.
Reutilización de consultas	Permiten reutilizar consultas complejas, lo que simplifica el proceso de obtener información específica.
Independencia de la estructura de la tabla	Se mantienen independientes de la estructura de la tabla subyacente, por lo que cambios en la estructura de la tabla no afectan la vista.
Mejora de rendimiento	Pueden mejorar el rendimiento al permitir consultar los datos de una manera más eficiente.
Actualización	Su contenido se actualiza automáticamente cuando los datos en las tablas subyacentes cambien, permitiendo una visualización actualizada de la información.

Resumiendo, son útiles para simplificar el acceso a los datos, mejorar la organización, mejorar la seguridad y mejorar el rendimiento en el acceso a los datos.

9.3 Ejemplo de cómo crear una vista con SQL

Siguiendo con el caso práctico de InkArt Studio, veamos el funcionamiento y utilidad de las vistas.

En InkArt Studio el personal consulta cada mañana el calendario completo de citas para saber quién atiende a cada cliente y cuándo. Hasta ahora, cada informe repite mentalmente “vamos a unir la tabla de citas con la de artistas y la de clientes, luego filtrar por fecha y ordenar…”, lo que hace que los informes sean largos de describir, lentos de redactar y propensos a errores.

¿Cómo definir un "acceso preparado" a esa información (nombre de artista, nombre de cliente y fecha) de forma que baste con pedir el nombre de la vista y obtener el listado correcto, ordenado y filtrado?

Solución (conceptual)

1. Se crea un objeto llamado **VistaAgenda** que internamente sabe "unir citas, artistas y clientes".
2. Cada vez que se consulta VistaAgenda, el sistema:
   - Aglutina automáticamente los datos de las tres tablas.
   - Filtra para mostrar solo cita, artista y cliente.
   - Ordena la información por artista y luego por fecha.
3. Gracias a esto, cualquier informe o pantalla que necesite el calendario solo escribe "muéstrame VistaAgenda" y obtiene el resultado completo, sin volver a describir las uniones ni los filtros.
4. Si mañana cambia el formato (por ejemplo, queremos ocultar al cliente que ya no es VIP), basta con modificar la definición de VistaAgenda y todas las aplicaciones que la usan reflejarán el cambio al instante.

Con este enfoque:

- **Se simplifica** la redacción de informes y interfaces.
- **Se reduce el riesgo de error**, al no repetir lógica compleja en cada consulta.
- **Se centraliza el mantenimiento**, porque sólo hay que actualizar la vista y no decenas de pantallas distintas.
- **Se mejora la seguridad**, ya que se puede dar acceso a VistaAgenda sin exponer otras columnas confidenciales.

Una vez que se aprenda SQL, esta operación se consigue con algo como:

```
CREATE VIEW VistaAgenda AS
SELECT A.Nombre AS Artista,
    C.Nombre AS Cliente,
    C.FechaHora
 FROM Citas C
```

```
 JOIN Artistas A ON C.ArtistaID = A.ArtistaID
 JOIN Clientes  C2 ON C.ClienteID = C2.ClienteID
ORDER BY A.Nombre, C.FechaHora;
```

Y luego, para ver los datos:

```
SELECT * FROM VistaAgenda;
```

10. Gestión de la seguridad

10.1 Introducción

La **gestión de seguridad** en un **Sistema de Gestión de Bases de Datos (SGBD)** incluye la administración de usuarios, roles, privilegios y límites en el uso del SGBD.

Componente	Descripción
Usuarios	Son individuos o aplicaciones que tienen acceso a la base de datos. Cada usuario tiene un nombre de usuario y una contraseña que se utilizan para autenticarse al acceder a la base de datos.
Roles	Son grupos de usuarios que comparten privilegios y responsabilidades. Por ejemplo, un rol de administrador puede tener permisos para crear y modificar tablas, mientras que un rol de usuario limitado solo puede leer datos.
Privilegios	Son permisos específicos que se otorgan a un usuario o a un rol para realizar ciertas acciones en la base de datos, como seleccionar, insertar, actualizar o eliminar datos.
Límites en el uso del SGBD	Los perfiles son conjuntos de restricciones que se imponen a un usuario o a un rol para controlar su uso de la base de datos. Por ejemplo, se pueden establecer límites en la cantidad de recursos que un usuario puede usar o en el tiempo de ejecución permitido para una consulta.

Es imprescindible una buena gestión de seguridad para garantizar la integridad y privacidad de los datos almacenados en la base de datos.

10.2 Usuarios

Los **usuarios de la base de datos** son individuos o aplicaciones que tienen acceso a la base de datos y pueden realizar operaciones en ella como consultar, insertar, actualizar o eliminar datos.

Para acceder a la base de datos, cada usuario debe tener un nombre de usuario y una contraseña que se utilizan para autenticarse.

Además de la autenticación, pueden ser asignados a roles o grupos, que comparten privilegios y responsabilidades. Por ejemplo, un rol de contable puede tener permisos para crear y modificar tablas relacionadas con la contabilidad, mientras que un rol de comercial solo podrá consultar algunos datos, como el límite de crédito de un cliente.

Es necesaria una buena gestión de usuarios para controlar quién tiene acceso a la base de datos y qué acciones pueden realizar en ella.

10.3 Roles

Los roles en la base de datos son grupos de usuarios que comparten privilegios y responsabilidades, que se utilizan para asignar permisos a múltiples usuarios de manera eficiente, en lugar de tener que asignar permisos a cada usuario individualmente.

Los roles también pueden ser utilizados para controlar el acceso a la base de datos. Por ejemplo, se puede restringir el acceso a ciertas tablas o vistas solo a usuarios que pertenecen a un rol específico.

Otro ejemplo sería que se puede crear un rol de " contable" que tenga permisos para crear y modificar tablas relacionadas con la contabilidad, y asignar ese rol a los usuarios que desempeñan el papel de contables en la organización. De esta manera, todos los usuarios asignados a ese rol tendrán los mismos permisos, sin tener que asignarlos de manera individual. Cuando entre un nuevo contable en la empresa, solamente habrá que asignarlo al rol correspondiente, sin tener que concretar los permisos específicos.

10.4 Privilegios

Los **privilegios en una base de datos** son permisos específicos que se otorgan a un usuario o a un rol para realizar acciones específicas en la base de datos, como leer, escribir, modificar, eliminar datos, crear tablas, ejecutar procedimientos almacenados, entre otros.

Los privilegios se utilizan para controlar el acceso a la información y para asegurar la integridad y seguridad de la base de datos.

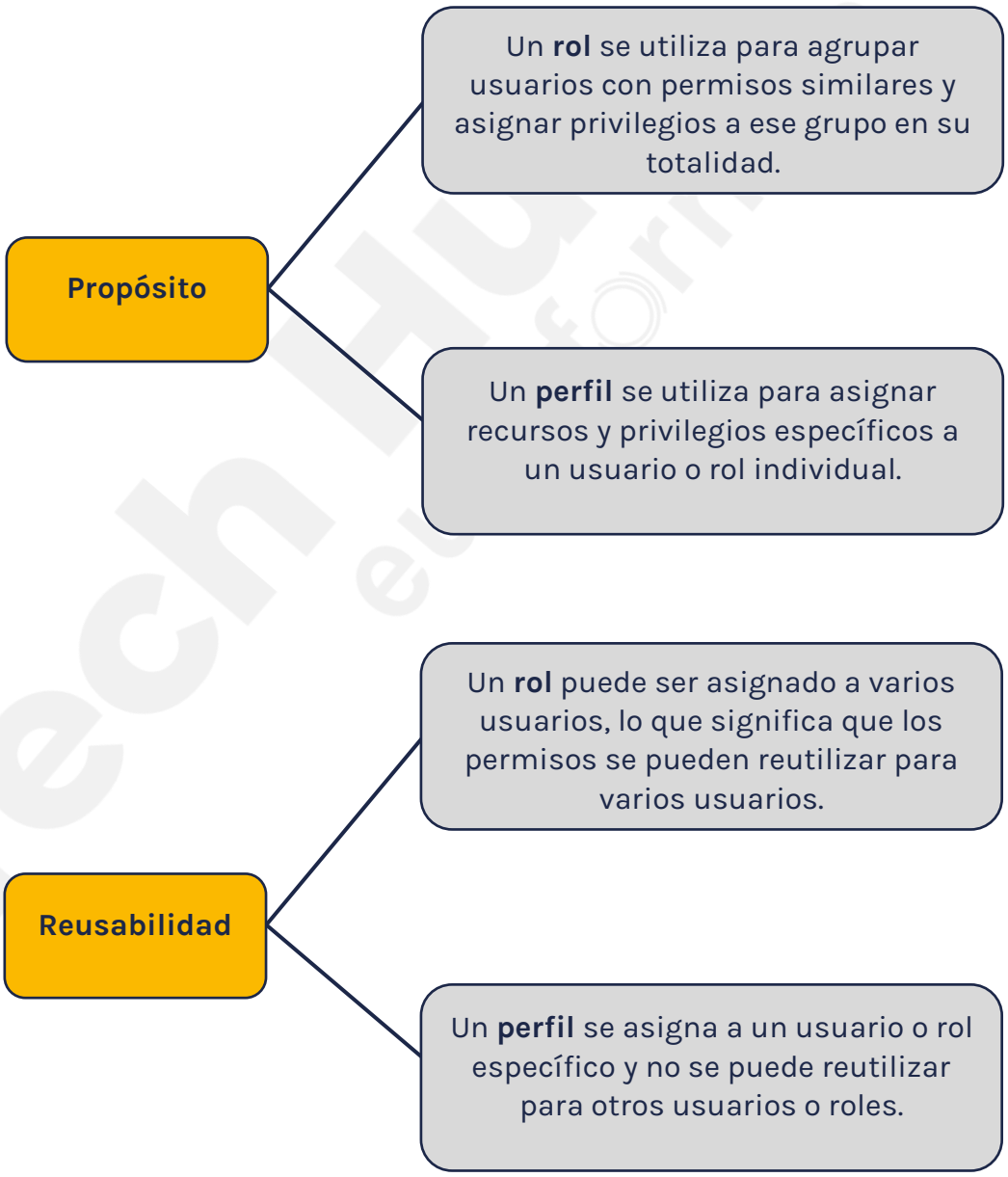
10.5 Perfiles

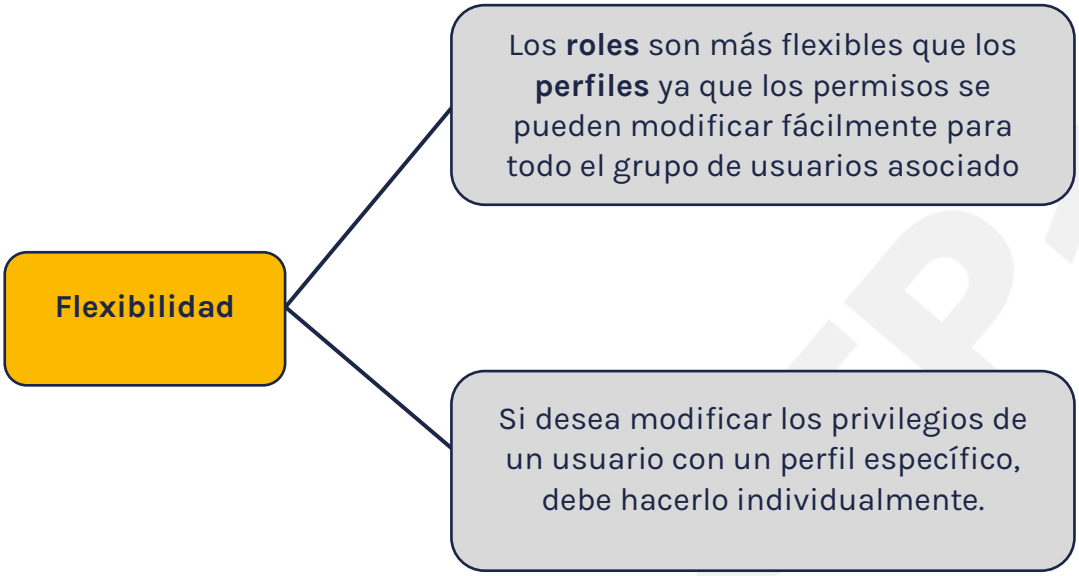
Un **perfil en una base de datos** es un conjunto de privilegios y recursos que se asignan a un usuario o rol específico. Los perfiles se utilizan para controlar el acceso de los usuarios a los recursos de la base de datos, como tablas, vistas, procedimientos almacenados y funciones.

Con los perfiles, los administradores de la base de datos pueden limitar los privilegios y recursos a los que los usuarios tienen acceso, lo que ayuda a garantizar la seguridad e integridad de la base de datos.

10.6 Diferencias entre roles y perfiles

Un **rol** y un **perfil en una base de datos** son conceptos similares, pero con algunas diferencias importantes:





En definitiva, los roles se utilizan para agrupar usuarios con permisos similares y los perfiles se utilizan para asignar privilegios específicos a un usuario o rol individual.

Veamos estos conceptos en el siguiente ejemplo práctico – Gestión de la seguridad en InkArt Studio

María es la administradora de la base de datos de InkArt Studio. El equipo está formado por Ana, la recepcionista; Luis, uno de los tatuadores; y Marta, la contable. Para proteger la información y facilitar el trabajo de cada uno, es necesario definir usuarios, roles, privilegios y perfiles que limiten el acceso según las responsabilidades de cada puesto.

¿Cómo se deben crear y organizar los usuarios, roles, privilegios y perfiles para que Ana solo pueda gestionar citas, Luis solo vea su propio calendario y actualice los detalles de sus tatuajes, y Marta consulte únicamente la facturación, sin poner en riesgo la integridad de la base de datos?

Solución

1. Creación de usuarios

Primero se crean tres cuentas de usuario en el SGBD, una para cada empleada:

- usuario_recepcion (Ana)
- usuario_artista (Luis)

- usuario_contable (Marta)

2. Definición de roles

A continuación, se definen tres roles que agrupan permisos según las funciones:

- Rol_Reception para Ana, con permisos de lectura y escritura sobre la tabla de Citas.
- Rol_Artist para Luis, con permisos de lectura sobre Citas y escritura sobre la tabla DetalleCita (para actualizar el estado y las notas de sus tatuajes).
- Rol_Accounting para Marta, con permisos de lectura sobre Clientes, Citas y Facturas.

3. Asignación de privilegios

Cada rol recibe únicamente los privilegios necesarios:

- El rol de recepción obtiene INSERT y UPDATE en Citas, y SELECT en Clientes para comprobar los datos de quien reserva.
- El rol de artista recibe SELECT en Citas y Clientes, y INSERT/UPDATE en DetalleCita.
- El rol de contabilidad solo tiene SELECT en Citas, Clientes y Facturas, de modo que pueda generar listados de ingresos sin alterar registros.

4. Creación de perfiles

Para evitar el uso abusivo de recursos o conexiones muy largas, se configura un perfil común que limite el tiempo de inactividad y el número máximo de sesiones simultáneas:

- Máximo de 2 sesiones por usuario.
- Tiempo de inactividad máximo de 15 minutos, tras el cual la sesión se cierra automáticamente.

5. Asignación de roles y perfiles a usuarios

Finalmente, cada usuario recibe su rol y el perfil de recursos:

- usuario_recepcion → Rol_Reception + Perfil_Estandar
- usuario_artista → Rol_Artist + Perfil_Estandar
- usuario_contable → Rol_Accounting+ Perfil_Estandar

Con esta configuración, Ana solo puede crear y modificar citas sin acceder a facturación, Luis solo ve las citas y puede anotar avances en sus tatuajes, y Marta accede a la información económica sin riesgo de alterar otros datos. El perfil de recursos garantiza que ninguna sesión permanezca abierta indefinidamente, manteniendo controlado el uso del SGBD.

En SQL, estas acciones se traducirían en sentencias como:

```
-- Creación de roles
CREATE ROLE Rol_Reception;
CREATE ROLE Rol_Artist;
CREATE ROLE Rol_Accounting;

-- Privilegios del rol de recepción
GRANT SELECT, INSERT, UPDATE ON Citas TO Rol_Reception;
GRANT SELECT ON Clientes TO Rol_Reception;

-- Privilegios del rol de artista
GRANT SELECT ON Citas, Clientes TO Rol_Artist;
GRANT INSERT, UPDATE ON DetalleCita TO Rol_Artist;

-- Privilegios del rol de contabilidad
GRANT SELECT ON Citas, Clientes, Facturas TO Rol_Accounting;

-- Creación de usuarios y asignación de roles
CREATE USER usuario_recepcion IDENTIFIED BY '*****';
GRANT Rol_Reception TO usuario_recepcion;
```

```
CREATE USER usuario_artista IDENTIFIED BY '*****';
GRANT Rol_Artist TO usuario_artista;

CREATE USER usuario_contable IDENTIFIED BY '*****';
GRANT Rol_Accounting TO usuario_contable;

-- Definición de perfil de recursos (ejemplo en Oracle)
CREATE PROFILE Perfil_Estandar LIMIT
  SESSIONS_PER_USER 2
  IDLE_TIME 15;
ALTER USER usuario_recepcion PROFILE Perfil_Estandar;
ALTER USER usuario_artista   PROFILE Perfil_Estandar;
ALTER USER usuario_contable  PROFILE Perfil_Estandar;
```

11. Lenguaje de definición de datos (DML, Data Manipulation Languaje)

11.1 Concepto

El Lenguaje de Definición de Datos (DDL) es el conjunto de instrucciones que sirven para **dar forma** a la base de datos antes de usarla. Con DDL se decide:

- **Qué "cajones" o tablas** habrá, es decir, dónde se guardarán los distintos tipos de información (clientes, pedidos, productos…).
- **Cómo serán esos cajones**, definiendo qué "casillas" o columnas tendrá cada uno (por ejemplo, nombre, fecha, cantidad).
- **Qué separadores o índices** adicionales se necesitan para encontrar datos con rapidez (como etiquetas que ayuden a buscar más rápido).
- **Qué "ventanas" o vistas** se crearán para ver la información filtrada o combinada de varias tablas.

Todos estos comandos (crear, cambiar o borrar estructuras) **modifican la forma de la base de datos de manera permanente**. Por eso, antes de ejecutar una instrucción DDL es fundamental planificar bien los cambios: una vez eliminada una tabla o columna, la estructura desaparece y puede ser muy difícil recuperarla sin copia de seguridad.

11.2 Comandos DDL

El Lenguaje de Definición de Datos (DDL) incluye las instrucciones que sirven para **construir y modificar la "estructura"** de la base de datos. A continuación, se describen los comandos más comunes, explicando en cada caso **qué hacen**:

- **CREATE TABLE**

Sirve para crear una nueva tabla y definir sus columnas y tipos de datos.

Ejemplo: para guardar clientes de una tienda se podría escribir:

```
Clientes (
ClienteID INT PRIMARY KEY,
Nombre VARCHAR(100),
Email VARCHAR(100)
)
```

- **ALTER TABLE**

Permite modificar la estructura de una tabla ya existente. Se usa para añadir o quitar columnas, cambiar tipos de datos, etc.

Ejemplo: si más tarde necesitamos el teléfono de cada cliente, haríamos: En la tabla Clientes, añade una columna Teléfono de tipo VARCHAR(20).

- **DROP TABLE**

Elimina por completo una tabla y todos sus datos. Es permanente y peligroso si no se ha hecho copia de seguridad.

Ejemplo: si existía una tabla temporal llamada ClientesAntiguos que ya no se usa, se ejecuta:
Elimina la tabla ClientesAntiguos de la base de datos.

- **CREATE INDEX**

Crea un índice sobre una o varias columnas para acelerar las búsquedas. Los índices ocupan espacio en disco pero reducen el tiempo de consulta.

Ejemplo: si se busca muy a menudo clientes por su email, se crea un índice: En la tabla Clientes, crea un índice sobre la columna Email para que las búsquedas por correo sean instantáneas.

- **DROP INDEX**

Quita un índice existente. Suele hacerse cuando el índice ya no aporta mejora o se quiere liberar espacio.

Ejemplo: tras comprobar que las búsquedas por email ya no son críticas, se elimina el índice en Clientes.Email.

- **CREATE VIEW**

Define una vista, es decir, un "alias" que combina y filtra datos de una o varias tablas sin duplicar información. Cada vez que se consulta, el SGBD reconstruye el resultado.

Ejemplo: para mostrar solo clientes activos con cita próxima: Crea la vista ClientesActivos que une Clientes y Citas filtrando fecha futura y estado "activo".

- **DROP VIEW**

Elimina una vista, borrando su definición de la base de datos. Los datos originales permanecen intactos.

Ejemplo: si ya no se necesita la vista ClientesActivos, se ejecuta: Elimina la vista ClientesActivos.

- **CREATE PROCEDURE**

Crea un procedimiento almacenado, un conjunto de instrucciones SQL guardadas bajo un nombre, que puede recibir parámetros y ejecutarse como una sola operación.

Ejemplo: para simplificar la inserción de citas, se podría crear un procedimiento "CrearCita" que reciba ClienteID, ArtistaID y FechaHora, y lo haga todo en bloque.

- **DROP PROCEDURE**

Elimina un procedimiento almacenado existente.

Ejemplo: si la lógica de "CrearCita" cambia y se redefine de otro modo, primero se ejecuta:
Elimina el procedimiento CrearCita para después crear la nueva versión.

Debemos tener en cuenta que **todos estos comandos DDL son permanentes**: una vez creados, modificados o eliminados los objetos de la base de datos, el cambio afecta inmediatamente a la estructura y no puede deshacerse con un simple "undo". Por ello siempre conviene revisar dos veces y, en entornos de producción, realizar copias de seguridad antes de aplicar alteraciones de DDL.

11.3 Buenas prácticas y gestión de cambios de esquema (DDL)

Al trabajar con comandos DDL, lo primero es asegurarse de contar con un entorno de pruebas idéntico al de producción. Antes de crear, modificar o eliminar cualquier tabla, vista o índice, conviene ejecutar los scripts en ese entorno de test para validar que no aparecen errores inesperados ni se rompen dependencias existentes.

Justo antes de aplicar un cambio, se recomienda realizar una copia de seguridad de la estructura —como mínimo de las tablas sobre las que vamos a actuar— de manera que, si algo falla, sea posible restaurar rápidamente el esquema anterior. No basta con confiar en el "deshacer": los comandos DDL son definitivos, así que planificar con antelación y programar las ventanas de mantenimiento en horas de baja actividad minimiza riesgos.

Cada modificación debe quedar registrada en un documento de cambios o "changelog", donde se anote la fecha, el responsable, el motivo del cambio y las comprobaciones realizadas. Esta documentación aporta transparencia y facilita la auditoría, además de servir como guía para futuros desarrollos.

Para gestionar versiones del esquema de forma profesional, conviene almacenar todos los scripts DDL en un sistema de control de versiones (por ejemplo, Git). De ese modo, cada script va numerado y descrito —por ejemplo, V1__crear_tabla_clientes.sql, V2__añadir_campo_telefono.sql—, lo cual permite reconstruir paso a paso cualquier versión anterior y entender la evolución de la base de datos.

En proyectos de cierta envergadura, se suele emplear una herramienta de migración como Flyway o Liquibase. Estas soluciones aplican automáticamente cada script en el orden correcto, registran en la propia base de datos qué migraciones ya se han ejecutado y ofrecen mecanismos para revertir cambios en caso de error. Integrar esta ejecución de migraciones en la canalización de integración continua (CI/CD) garantiza que, al desplegar

nueva versión del software, el esquema siempre se actualiza de forma coherente y sin olvidos manuales.

Finalmente, es importante prever la posibilidad de "rollback": cada migración que elimina o renombra objetos debe venir acompañada de su script inverso o de un procedimiento documentado para restaurar la estructura previa. De esta manera, se mantiene la agilidad para evolucionar el modelo de datos, sin renunciar a la seguridad de poder deshacer cambios complejos si fuera necesario.

Ejemplo – Evolución del esquema de InkArt Studio con DDL

InkArt Studio ha decidido incorporar un nuevo servicio de "Cover-up" (tatuajes de cubrimiento) y para ello es necesario adaptar la base de datos. Hasta ahora sólo se gestionaban citas y diseños estándar; con este cambio habrá que crear nuevas tablas, modificar otras, añadir índices y exponer vistas específicas. Dado que estos cambios son definitivos y afectan a la estructura de producción, conviene seguir un proceso riguroso.

¿Cómo planificar y aplicar con seguridad los cambios de DDL necesarios —crear la tabla CoverUp, alterar la tabla Citas, añadir índices, definir una vista para las cubiertas— de forma que no interrumpa el servicio ni introduzca errores?

Solución

Antes de nada, todos los scripts de DDL se validan en el entorno de pruebas, que es una réplica exacta de la base de datos de producción. Allí se ejecuta primero un respaldo de las tablas implicadas ("Citas" y cualquier otra que pueda verse afectada) para poder restaurarlas si algo falla.

A continuación, se crea un archivo de migración numerado (V3__crear_tabla_coverup.sql) en el repositorio Git que contiene el script DDL para construir la nueva tabla CoverUp con sus columnas (ID autónomo, descripción del diseño, precio, duración). Mientras tanto, otro script (V4__alter_citas_add_coverup_id.sql) altera la tabla Citas para añadir la columna CoverUpID, definiendo esta nueva clave foránea con su índice asociado para acelerar las búsquedas de citas de tipo cover-up.

Cada uno de estos cambios se documenta en el changelog interno: fecha, autor, motivo ("nueva funcionalidad Cover-up"), y los resultados de las pruebas en staging. Después, se crea un índice B-tree sobre CoverUpID para mejorar el rendimiento de los listados de este servicio, y en paralelo se define una vista AgendaCoverUp que combina CoverUp, Citas, Clientes y Artistas, mostrando solo las sesiones de cubrimiento próximas y permitiendo a recepción y diseño consultarlas con un simple SELECT.

Una vez validados los scripts en pruebas (incluidas comprobaciones de integridad referencial y tiempos de ejecución), se ejecutan en producción durante la ventana de mantenimiento pactada. Se registra en la propia base de datos, mediante la herramienta de migración (por ejemplo, Flyway), que V3 y V4 ya han sido aplicados, lo que evita re-ejecuciones accidentales.

De igual modo, se ha preparado un "rollback" (R3__drop_coverup.sql) que desecha en orden inverso todas las modificaciones si fuera necesario restaurar la estructura previa.

Gracias a este proceso:

- La producción sufre un corte mínimo (solo la ventana de mantenimiento).
- Cada cambio va numerado y versionado.
- Las copias de seguridad previas permiten deshacer cualquier error grave.
- El changelog y los scripts en Git ofrecen trazabilidad y ayudan a futuros desarrollos.
- La vista AgendaCoverUp pone a disposición un acceso sencillo y seguro a la nueva funcionalidad sin repetir joins ni filtros en cada consulta.

Con este enfoque, InkArt Studio evoluciona su base de datos de forma ordenada, segura y documentada, cumpliendo con las mejores prácticas de DDL y gestión de cambios de esquema.

A modo de ilustración, se muestran los comandos que utilizaríamos en mysql para aplicar los cambios descritos en el caso práctico:

```
-- 1. Crear la tabla CoverUp
CREATE TABLE IF NOT EXISTS CoverUp (
  CoverUpID     INT AUTO_INCREMENT PRIMARY KEY,
  Descripcion   VARCHAR(200) NOT NULL,
  Precio        DECIMAL(6,2) NOT NULL,
  DuracionMin   INT NOT NULL
) ENGINE=InnoDB;

-- 2. Alterar la tabla Citas para añadir la clave foránea a CoverUp
ALTER TABLE Citas
  ADD COLUMN CoverUpID INT,
  ADD CONSTRAINT fk_citas_coverup
    FOREIGN KEY (CoverUpID) REFERENCES CoverUp(CoverUpID)
    ON DELETE SET NULL
    ON UPDATE CASCADE;

-- 3. Crear un índice para acelerar las búsquedas por CoverUpID
CREATE INDEX idx_citas_coverup ON Citas(CoverUpID);

-- 4. Definir o actualizar la vista AgendaCoverUp
CREATE OR REPLACE VIEW AgendaCoverUp AS
  SELECT
    A.Nombre      AS Artista,
    CL.Nombre     AS Cliente,
    C.FechaHora   AS Fecha,
    CU.Descripcion AS TipoCoverUp
  FROM Citas C
  JOIN Artistas A   ON C.ArtistaID = A.ArtistaID
  JOIN Clientes CL  ON C.ClienteID  = CL.ClienteID
```

```
LEFT JOIN CoverUp CU ON C.CoverUpID = CU.CoverUpID
WHERE C.CoverUpID IS NOT NULL
ORDER BY A.Nombre, C.FechaHora;
```

12. Lenguaje de manipulación de datos (DCL, Data Control Language)

12.1 Concepto

El **Lenguaje de Manipulación de Datos** (**DML**, **Data Manipulation Language**) es un conjunto de comandos utilizados para manipular y acceder a los datos almacenados en una base de datos. Los comandos DML permiten insertar, actualizar, eliminar y recuperar datos en una base de datos.

El Lenguaje de Manipulación de Datos (DML) es la parte del SQL que se ocupa **exclusivamente** del contenido de la base de datos, es decir, de los registros que viven dentro de las tablas, sin cambiar nunca su estructura. Mientras que el DDL prepara los "cajones" donde se guardarán los datos, el DML es el conjunto de acciones que permiten **llenarlos**, **modificarlos**, **consultarlos** o **vaciar** porciones de ellos.

En la práctica diaria, el DML se compone de **cuatro operaciones** fundamentales:

- **INSERT** para añadir nuevos registros. Por ejemplo, al incorporar un nuevo cliente a la base de InkArt Studio, se usa INSERT para crear su fila en la tabla Clientes con su nombre, email y teléfono.
- **UPDATE** para modificar valores existentes. Cuando un cliente cambia de dirección o un tatuador corrige el precio de un diseño, UPDATE actualiza únicamente esas celdas, dejando el resto del registro intacto.
- **DELETE** para eliminar registros que ya no son necesarios. Si un cliente se da de baja o un tatuaje queda cancelado, DELETE quita la fila correspondiente de la tabla, evitando que permanezca información obsoleta.
- **SELECT** para recuperar información y presentarla al usuario. Cada consulta que muestra la lista de citas próximas o filtra los clientes con saldo pendiente está basada en un SELECT que extrae, ordena y, si es necesario, relaciona varias tablas.

A diferencia de los comandos DDL, las sentencias DML **no alteran la estructura** de los objetos, y suelen ejecutarse en tiempo de ejecución de la aplicación. En la mayoría de los SGBD, estas operaciones pueden agruparse dentro de transacciones (tema que se verá más adelante) para asegurar que,

si algo falla al insertar o actualizar varios registros, sea posible deshacer todos los cambios de golpe y mantener la integridad de los datos.

12.2 Comandos DML

El DML agrupa tanto las operaciones de manipulación de registros como las de control de transacciones. A continuación, se describen y ejemplifican los principales comandos:

Manipulación de registros

Conjunto de instrucciones que permiten crear, leer, actualizar y borrar filas en las tablas:

- **SELECT**
  Recupera datos de una o varias tablas. Por ejemplo, para mostrar el nombre y correo de clientes activos se podría conceptualizar:

  "Obtener Nombre, Email de la tabla Clientes donde Estado = 'Activo'."

  Esta instrucción no modifica nada: solo lee y presenta la información.

- **INSERT**
  Añade nuevos registros. En InkArt Studio, el ejemplo que hemos trabajado a lo largo de toda la unidad, al registrar una nueva cita se usaría:

  "Insertar en Citas los valores ClienteID, ArtistaID y FechaHora."

  Permite poblar las tablas con la información que llega día a día.

- **UPDATE**
  Modifica valores existentes en uno o varios registros.

  Por ejemplo: "Actualizar el teléfono de un cliente con ClienteID = 7."

  Solo cambia las columnas indicadas, dejando intactos el resto de los datos de esa fila.

- **DELETE**
  Elimina registros que ya no son necesarios. Para borrar una cita cancelada:
  "Eliminar de Citas donde CitaID = 102."

  Retira la fila por completo, evitando que datos obsoletos permanezcan en la base de datos.

- **MERGE**
  Fusiona datos de dos tablas o de una tabla de "staging" en la tabla definitiva, insertando nuevas filas y actualizando existentes según una condición.
  Ejemplo simplificado:

  "Combinar registros de la tabla ImportacionClientes con Clientes: si el cliente existe actualizar su email; si no, insertarlo."

- **TRUNCATE TABLE**
  Vacía una tabla completa con un único comando, eliminando todas las filas de forma rápida pero manteniendo su definición. En un escenario de pruebas, para limpiar por completo la tabla temporal de importación se podría usar:

  "Vaciar ImportacionCoberturas" sin afectar la estructura ni los índices.

Control de transacciones

Permite agrupar varias operaciones DML en una única unidad de trabajo atómica:

- **SAVEPOINT**
  Marca un punto intermedio dentro de la transacción. Si a continuación algo falla, se puede volver a este punto sin deshacer todo.

  Ejemplo conceptual:

  "Dentro de la transacción, guardar punto 'Paso1' tras insertar la cabecera de un pedido."

- **ROLLBACK**
  Deshace los cambios realizados desde el inicio de la transacción o desde el último savepoint.

  Tras un error al insertar varios detalles de cita, se puede "revertir al savepoint 'Paso1'" para conservar solo lo anterior.

- **COMMIT**
  Confirma definitivamente todos los cambios realizados en la transacción.

  Una vez comprobado que la inserción de cita y detalles ha sido correcta, se emite "commit" para que los datos pasen a "persistente" y sean visibles para otros usuarios.

Así, los comandos **SELECT**, **INSERT**, **UPDATE**, **DELETE**, **MERGE** y **TRUNCATE** definen el día a día de la manipulación de registros, mientras que **SAVEPOINT**, **ROLLBACK** y **COMMIT** ofrecen el control necesario para asegurar que cada conjunto de operaciones sea seguro y coherente.

12.3 Ejemplo de comandos de manipulación de datos

A continuación, se muestra cómo, en la tabla de citas (Citas), se pueden insertar, actualizar, borrar y consultar registros reales de InkArt Studio:

1. INSERT: Agregar una nueva cita para el cliente "María Gómez"

```
INSERT INTO Citas (ClienteID, ArtistaID, FechaHora, Estado)
VALUES (5, 2, '2025-08-01 10:00:00', 'Agendada');
```

Este **INSERT** añade una nueva fila a la tabla **Citas**, es decir, crea una cita para "María Gómez" con los datos que se indican:

1. Tabla: Citas

2.-Columnas que se rellenan:

- ClienteID → 5 (corresponde a "María Gómez")
- ArtistaID → 2 (identifica al tatuador encargado)
- FechaHora → '2025-08-01 10:00:00' (día y hora de la sesión)
- Estado → 'Agendada' (marca que la cita está programada)

Al ejecutar esta sentencia, la base de datos reserva ese "espacio" en **Citas** con esos cuatro valores, dejando listo el registro para posteriores actualizaciones o consultas.

2. UPDATE: Cambiar el estado de la cita cuando el tatuador marca "finalizada"

```
UPDATE Citas
SET Estado = 'Finalizada'
WHERE ClienteID = 5
```

```
AND ArtistaID = 2
AND FechaHora = '2025-08-01 10:00:00';
```

Esta instrucción **UPDATE** modifica el valor de la columna **Estado** en la tabla **Citas**, pero solo para la cita concreta que coincide con todos los criterios del **WHERE**:

1.-UPDATE Citas

Indica que la operación afectará a la tabla **Citas**.

2.-SET Estado = 'Finalizada'

Cambia el campo **Estado** de los registros seleccionados, asignándole el valor **'Finalizada'**.

3.-WHERE ClienteID = 5 AND ArtistaID = 2 AND FechaHora = '2025-08-01 10:00:00'
Filtra para que solo se actualice la fila que tiene:

- **ClienteID** igual a **5** (María Gómez),
- **ArtistaID** igual a **2** (el tatuador asignado),
- **FechaHora** exactamente **'2025-08-01 10:00:00'** (la cita agendada).

En conjunto, el comando busca ese único registro y le pone el estado **'Finalizada'**, reflejando que el tatuaje ha sido completado.

3. DELETE: Eliminar la cita si el cliente la cancela

```
DELETE FROM Citas
WHERE ClienteID = 5
AND ArtistaID = 2
AND FechaHora = '2025-08-01 10:00:00'
AND Estado = 'Cancelada';
```

Esta sentencia **DELETE** borra de la tabla **Citas** únicamente el registro que cumpla **todas** las condiciones del filtro:

1.-DELETE FROM Citas

Indica que la operación eliminará filas de la tabla **Citas**.

2.-WHERE ClienteID = 5

Solo considera las citas cuyo **ClienteID** sea **5** (María Gómez).

3.-AND ArtistaID = 2

Restringe aún más para el tatuador con **ArtistaID 2**.

4.-AND FechaHora = '2025-08-01 10:00:00'

Se asegura de apuntar a la cita programada en esa fecha y hora exactas.

5.-AND Estado = 'Cancelada'

Solo procede si, además, el **Estado** de la cita ya está marcado como **'Cancelada'**.

En conjunto, el comando elimina esa única fila de la tabla, retirando completamente la cita cancelada y evitando dejar registros obsoletos.

4. SELECT: Recuperar las próximas 10 citas de hoy en adelante

```
SELECT C.FechaHora,
    CL.Nombre   AS Cliente,
    A.Nombre    AS Artista,
    C.Estado
 FROM Citas C
 JOIN Clientes CL  ON C.ClienteID = CL.ClienteID
 JOIN Artistas A   ON C.ArtistaID = A.ArtistaID
 WHERE C.FechaHora >= CURRENT_TIMESTAMP
 ORDER BY C.FechaHora
 LIMIT 10;
```

Esta consulta **SELECT** extrae un listado de las próximas citas, mostrando fecha, cliente, artista y estado, y limita el resultado a las 10 más cercanas. Desglosemos cada parte:

1.-SELECT C.FechaHora, CL.Nombre AS Cliente, A.Nombre AS Artista, C.Estado

- Indica las **columnas** que se desean recuperar:
  - C.FechaHora: la fecha y hora de la cita.
  - CL.Nombre AS Cliente: el nombre del cliente (renombrado para legibilidad).

- A.Nombre AS Artista: el nombre del tatuador (también renombrado).
- C.Estado: el estado actual de la cita (por ejemplo, "Agendada" o "Finalizada").

2.-FROM Citas C

- Define la **tabla principal** de la que se extraen los datos, Citas, y le asigna el alias C para referirse a ella de forma abreviada.

3.-JOIN Clientes CL ON C.ClienteID = CL.ClienteID

- Realiza una **unión interna** con la tabla Clientes (alias CL), vinculando cada fila de Citas con su registro correspondiente en Clientes a través del campo ClienteID.

4.-JOIN Artistas A ON C.ArtistaID = A.ArtistaID

- Añade otra unión interna con Artistas (alias A), relacionando ArtistaID en Citas con el mismo campo en Artistas, para obtener el nombre del tatuador.

5.-WHERE C.FechaHora >= CURRENT_TIMESTAMP

- Filtra las filas para quedarse solo con las citas cuya fecha y hora sean **iguales o posteriores** al momento actual (CURRENT_TIMESTAMP), es decir, las citas futuras.

6.-ORDER BY C.FechaHora

- Ordena el resultado **de forma ascendente** por la fecha y hora de la cita, mostrando primero las más cercanas cronológicamente.

7.-LIMIT 10

- Restringe la salida a las **10 primeras filas** del conjunto ordenado, de modo que solo se vean las diez citas inmediatas por venir.

  En conjunto, esta sentencia recupera de forma eficiente y legible las próximas diez sesiones agendadas en InkArt Studio, combinando datos de tres tablas y aplicando filtro, orden y límite de resultados.

13. Lenguaje de control de datos (DCL, Data Control Language)

13.1 Concepto

El **Lenguaje de Control de Datos** (**DCL**, **Data Control Language**) es un conjunto de comandos utilizados para controlar el acceso y los permisos en una base de datos.

Los comandos DCL permiten establecer reglas y permisos para los usuarios y roles que interactúan con la base de datos.

El Lenguaje de Control de Datos (DCL) constituye el mecanismo de seguridad interno de un SGBD, encargado de definir y supervisar **quién** puede realizar **qué** operaciones sobre **qué** objetos. A través del DCL se conceden o revocan permisos granulares —lectura, escritura, modificación, eliminación, ejecución de procedimientos, creación o eliminación de objetos— tanto a usuarios individuales como a grupos (roles).

Este control de acceso se basa en el principio de mínimos privilegios: cada cuenta recibe únicamente los permisos estrictamente necesarios para desempeñar su función, minimizando así el riesgo de accesos no autorizados o modificaciones accidentales. El DCL habilita además la segregación de funciones, al permitir que distintas tareas (por ejemplo, gestión de datos, administración de esquemas, auditoría) se asignen a distintos perfiles con permisos independientes.

Además de la concesión inicial, el DCL contempla la posibilidad de ajustar dinámicamente estos permisos a medida que cambian las responsabilidades —por ejemplo, revocando un acceso de escritura cuando un rol deja de requerirlo— y de definir políticas de uso que impidan la escalada indebida de privilegios. Todo ello garantiza que la base de datos mantenga su integridad, confidencialidad y disponibilidad, adaptándose a los requerimientos de seguridad y cumplimiento normativo de la organización.

13.2 Comandos DCL

El DCL agrupa las operaciones destinadas a gestionar las cuentas de acceso, los roles y los permisos sobre los distintos objetos de la base de datos. A continuación se describen y ejemplifican los comandos más relevantes:

CREATE USER

Crea una nueva cuenta de usuario que podrá autenticarse y recibir privilegios.

Por ejemplo: "Crear el usuario usuario_ventas con contraseña Ventas2025 y esquema predeterminado Ventas."

ALTER USER

Modifica los atributos de un usuario existente sin eliminarlo ni afectar sus permisos.

Por ejemplo: "Cambiar la contraseña de usuario_ventas a Nuev@Pass123 y asignarle el perfil de recursos Perfil_Limite."

DROP USER

Elimina por completo una cuenta y sus privilegios, revocando cualquier acceso futuro al sistema.

Por ejemplo: "Borrar al usuario usuario_prueba porque ya no participa en ningún proyecto."

GRANT

Otorga privilegios específicos de lectura, escritura o administración sobre tablas, vistas, procedimientos u otros objetos, ya sea a un usuario o a un rol.

Por ejemplo: "Conceder a usuario_ventas permiso SELECT y INSERT sobre la tabla Pedidos."

REVOKE

Retira permisos previamente asignados con GRANT, garantizando el principio de mínimos privilegios.

Por ejemplo: "Revocar de usuario_ventas el permiso INSERT en Pedidos tras cerrar la campaña."

SET ROLE

Activa durante la sesión uno de los roles asignados al usuario, habilitando el conjunto de permisos asociados al mismo.

Por ejemplo: "Dentro de una sesión, cambiar al rol Rol_Reportes para ejecutar solo consultas de lectura."

13.3 Ejemplo de comandos de control de datos

Para ilustrar cómo se aplican los comandos DCL en un entorno real, veamos los pasos necesarios para dar de alta a la recepcionista de InkArt Studio, "Ana", y permitirle gestionar citas y clientes sin acceso a eliminaciones ni a la estructura de la base de datos. Se muestran las instrucciones tanto en **MySQL** como en **SQL Server**.

MySQL

1. Crear el usuario 'ana_recepcion' que solo podrá conectarse desde localhost

```
CREATE USER 'ana_recepcion'@'localhost' IDENTIFIED BY 'Recep2025!';
```

Este comando crea una nueva cuenta llamada **ana_recepcion** que solo podrá iniciar sesión desde el propio servidor (localhost). La cláusula IDENTIFIED BY asigna la contraseña **Recep2025!** a dicha cuenta. A partir de aquí, Ana ya existe como usuario de la base de datos, pero todavía no puede acceder a ningún dato hasta que se le concedan permisos.

2. Otorgar a Ana permisos de lectura y escritura en las tablas Citas y Clientes

```
GRANT SELECT, INSERT, UPDATE
 ON inkart_studio.Citas TO 'ana_recepcion'@'localhost';
GRANT SELECT, INSERT, UPDATE
 ON inkart_studio.Clientes TO 'ana_recepcion'@'localhost';
```

Mediante estos dos comandos **GRANT** se otorgan a Ana los privilegios necesarios para **consultar** (SELECT), **crear** (INSERT) y **modificar** (UPDATE) registros en las tablas **Citas** y **Clientes**, respectivamente. De este modo, podrá agendar y actualizar citas, así como agregar o corregir datos de clientes, pero no podrá borrar nada.

3. Más adelante, si Ana necesita también marcar citas como "finalizadas",

```
-- se le añade el permiso DELETE sobre Citas
GRANT DELETE
 ON inkart_studio.Citas TO 'ana_recepcion'@'localhost';
```

Si en el futuro se decide que Ana también debe poder eliminar citas (por ejemplo, para marcar aquellas que quedaron completamente finalizadas), este comando añade el permiso **DELETE** específicamente sobre la tabla **Citas**. A partir de entonces, Ana podrá borrar filas de esa tabla, siempre dentro de su conexión desde localhost.

4. Si fuera necesario retirarle el permiso de insertar clientes por un cambio de funciones:

```
REVOKE INSERT
 ON inkart_studio.Clientes FROM 'ana_recepcion'@'localhost';
```

En caso de que cambien sus responsabilidades y ya no necesite crear nuevos clientes, este comando **REVOKE** elimina únicamente el privilegio **INSERT** sobre la tabla **Clientes**. Ana conservará sus permisos de lectura y actualización en esa tabla, pero ya no podrá agregar registros nuevos.

SQL Server

1. Crear el login y el usuario en la base de datos inkart_studio

```
CREATE LOGIN ana_recepcion WITH PASSWORD = 'Recep2025!';
CREATE USER ana_recepcion FOR LOGIN ana_recepcion;
```

La primera línea crea a nivel de servidor un "login" llamado **ana_recepcion** y le asigna la contraseña **'Recep2025!'**. La segunda instrucción vincula ese login a un usuario de la base de datos **inkart_studio**, de modo que Ana pueda autenticar y, a continuación, operar dentro de esa base de datos.

2. Conceder permisos de lectura y escritura en Citas y Clientes

```
GRANT SELECT, INSERT, UPDATE
  ON Citas TO ana_recepcion;
GRANT SELECT, INSERT, UPDATE
  ON Clientes TO ana_recepcion;
```

3. Para permitir marcar citas como completadas:

```
GRANT DELETE
  ON Citas TO ana_recepcion;

-- 4. Para revocar la posibilidad de actualizar clientes:
REVOKE UPDATE
  ON Clientes FROM ana_recepcion;
```

14. Resumen

Un **modelo de datos** es una representación abstracta de los datos y sus relaciones dentro de un sistema informático.

Las **relaciones en un modelo relacional** representan la estructura de los datos en forma de tablas.

Cada **tabla** tiene un conjunto de **atributos**, que representan las características o propiedades de los datos que se almacenan en la tabla.

Una **tupla** es un registro o una fila en una tabla, que representa una instancia específica de los atributos en una relación.

Las relaciones, atributos y tuplas juntos forman la **base para la representación** y el almacenamiento de datos en un modelo relacional.

Los tipos de datos incluyen numéricos, cadenas, fechas y horas, y booleanos, además de otros tipos específicos definidos por cada base de datos.

Las **claves primarias** identifican de forma única una tupla dentro de una relación.

Los **índices** mejoran la velocidad de búsqueda de datos y pueden ser de diferentes tipos, como índices secuenciales o hash.

El **valor NULL** representa un valor desconocido o no aplicable en una columna de una tabla.

Las **claves ajenas** son un tipo de restricción en un modelo relacional que establece una relación entre dos tablas.

La clave ajena se define en una tabla secundaria y hace referencia a una clave primaria en una tabla primaria.

La relación entre las dos tablas se utiliza para garantizar la integridad referencial de los datos, lo que significa que los valores en la tabla secundaria deben coincidir con los valores en la tabla primaria. Esto impide la inserción de valores inválidos en la tabla secundaria y asegura que los datos estén relacionados de forma consistente en todas las tablas en el modelo de datos.

Las **vistas** son una representación virtual de los datos en una tabla que pueden ser personalizadas por el usuario.

La **gestión de seguridad** incluye la administración de usuarios, roles, privilegios y perfiles para controlar el acceso a los datos.

El **DDL (Lenguaje de Definición de Datos)** se utiliza para crear y modificar la estructura de las tablas en la base de datos.

El **DML (Lenguaje de Manipulación de Datos)** se utiliza para insertar, actualizar y eliminar datos.

El **DCL (Lenguaje de Control de Datos**) se utiliza para administrar permisos y para controlar la transacción de datos.

Tech Hub <FP> euroformac

Glosario

Modelo de Datos: Una representación abstracta de los datos y sus relaciones dentro de un sistema informático.

Relaciones: Una estructura de datos en un modelo relacional que se representa como una tabla.

Atributos: Las características o propiedades de los datos que se almacenan en una relación o tabla.

Tuplas: Un registro o fila en una tabla, que representa una instancia específica de los atributos en una relación.

Tipos de Datos: Categorías de valores que se utilizan para describir los atributos en una relación, como números, cadenas, fechas y horas, y booleanos.

Claves Primarias: Una restricción en un modelo relacional que identifica de forma única una tupla dentro de una relación.

Índices: Estructuras de datos que mejoran la velocidad de búsqueda de datos en una tabla.

Valor NULL: Representa un valor desconocido o no aplicable en una columna de una tabla.

Claves Ajenas: Una restricción en un modelo relacional que establece una relación entre dos tablas.

Vistas: Una representación virtual de los datos en una tabla que puede ser personalizada por el usuario.

Usuario: Individuo o entidad que accede a una base de datos.

Rol: Conjunto de permisos y responsabilidades asignados a un usuario o grupo de usuarios en una base de datos.

Perfil: Conjunto de atributos y preferencias asociados a un usuario específico en una base de datos.

Privilegios: Derechos específicos otorgados a un usuario o rol para realizar acciones específicas en una base de datos, como seleccionar, actualizar o eliminar datos.

Acceso: Derecho otorgado a un usuario o rol para acceder a una base de datos o recurso específico.

Control de acceso: Conjunto de medidas y procedimientos utilizados para garantizar que solo los usuarios autorizados tengan acceso a una base de datos o recurso.

DDL (Lenguaje de Definición de Datos): Un lenguaje utilizado para crear y modificar la estructura de las tablas en una base de datos.

DML (Lenguaje de Manipulación de Datos): Un lenguaje utilizado para insertar, actualizar y eliminar datos en una base de datos.

DCL (Lenguaje de Control de Datos): Un lenguaje utilizado para administrar permisos y controlar la transacción de datos en una base de datos.

Tech Hub <FP> euroformac

Bibliografía

García, R. (2000). Bases de datos: conceptos y arquitectura. Mc Graw Hill.

López, J. (2007). Modelos de bases de datos. Anaya Multimedia.

Martínez, M. (2010). Modelado de datos en bases de datos. Editorial Universitas.

Rodríguez, C. (2011). Desarrollo de aplicaciones con bases de datos. Prentice Hall.

Sánchez, J. (2009). Modelos y lenguajes para bases de datos. Ediciones ENI.